Contents

- Astronomical Coordinates 1: Getting Started with astropy.coordinates
- Astronomical Coordinates 2: Transforming Coordinate Systems and Representations
- Astronomical Coordinates 3: Working with Velocity Data in astropy.coordinates
- Astronomical Coordinates 4: Cross-matching Catalogs Using astropy.coordinates and astroquery

This is a set of tutorials on how to use the `astropy.coordinates` package to work with astronomical coordinates. We will introduce many of the key concepts underlying how to represent and transform astronomical coordinates using `astropy.coordinates`, including how to work with both position and velocity data within the coordinate objects. We will also explore how the `astropy.coordinates` package can be used to cross-match two catalogs that contain overlapping sources that may have been observed at different times.

Authors

Adrian Price-Whelan

Learning Goals

- Create `astropy.coordinates.SkyCoord` objects using coordinate data and object names
- Use SkyCoord objects to become familiar with object oriented programming (OOP)
- Use a `SkyCoord` object to query the *Gaia* archive using `astroquery`
- Output coordinate data in different string representations
- Demonstrate working with 3D sky coordinates (including distance information for objects)

Keywords

coordinates, OOP, astroquery, gaia

Summary

Astronomers use a wide variety of coordinate systems and formats to represent sky coordinates of celestial objects. For example, you may have seen terms like “right ascension” and “declination” or “galactic latitude and longitude,” and you may have seen angular coordinate components represented as “0h39m15.9s,” “00:39:15.9,” or 9.81625º. The subpackage `astropy.coordinates` provides tools for representing the coordinates of objects and transforming them between different systems.

In this tutorial, we will explore how the `astropy.coordinates` package can be used to work with astronomical coordinates. You may find it helpful to keep the Astropy documentation for the coordinates package open alongside this tutorial for reference or additional reading. In the text below, you may also see some links that look like (docs). These links will take you to parts of the documentation that are directly relevant to the cells from which they link.

Note: This is the 1st tutorial in a series of tutorials about astropy.coordinates.

Imports

We start by importing some packages that we will need below:

```
import matplotlib.pyplot as plt

%matplotlib inline
import numpy as np

from astropy import units as u
from astropy.coordinates import SkyCoord, Distance
from astropy.table import QTable

from astroquery.gaia import Gaia

Gaia.ROW_LIMIT = 10000  # Set the row limit for returned data
```

Representing On-sky Positions with `astropy.coordinates`

In Astropy, the most common way of representing and working with sky coordinates is to use the `SkyCoord` object (docs). A `SkyCoord` can be created directly from angles or arrays of angles with

associated units, as demonstrated below.

To get started, let's assume that we want to create a `SkyCoord` object for the center of the open star cluster NGC 188 so that later we can query and retrieve stars that might be members of the cluster. Let's also assume, for now, that we already know the sky coordinates of the cluster to be (12.11, 85.26) degrees in the ICRS coordinate frame. The ICRS — sometimes referred to as "equatorial" or "J2000" coordinates (more info about the ICRS) — is currently the most common astronomical coordinate frame for stellar or extragalactic astronomy, and is the default coordinate frame for `SkyCoord`. Since we already know the ICRS position of NGC 188 (see above), we can create a `SkyCoord` object for the cluster by passing the data in to the `SkyCoord` initializer:

```
ngc188_center = SkyCoord(12.11 * u.deg, 85.26 * u.deg)
ngc188_center
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (12.11, 85.26)>
```

Even though the default frame is ICRS, it is generally recommended to explicitly specify the frame your coordinates are in. In this case, this would be an equivalent way of creating our `SkyCoord` object for NGC 188:

```
ngc188_center = SkyCoord(12.11 * u.deg, 85.26 * u.deg, frame="icrs")
ngc188_center
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (12.11, 85.26)>
```

As we will see later on in this series, there are many other supported coordinate frames, so it helps to get into the habit of passing in the name of a coordinate frame.

In the above initializations, we passed in `astropy.units.Quantity` objects with angular units to specify the angular components of our sky coordinates. The `SkyCoord` initializer will also accept string-formatted coordinates either as separate strings for Right Ascension (RA) and Declination (Dec) or a single string. For example, if we have sexagesimal sky coordinate data: In this case, the representation of the data includes specifications of the units (the "hms" for "hour minute second", and the "dms" for "degrees minute second"

```
SkyCoord("00h48m26.4s", "85d15m36s", frame="icrs")
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (12.11, 85.26)>
```

Some string representations do not explicitly define units, so it is sometimes necessary to specify the units of the string coordinate data explicitly if they are not implicitly included:

```
SkyCoord("00:48:26.4 85:15:36", unit=(u.hour, u.deg), frame="icrs")
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (12.11, 85.26)>
```

For more information and examples on initializing `SkyCoord` objects, see this documentation.

For the `SkyCoord` initializations demonstrated above, we assumed that we already had the coordinate component values ready. If you do not know the coordinate values and the object you are interested in is in SESAME, you can also automatically look up and load coordinate values from the name of the object using the `SkyCoord.from_name()` class method[1] (docs). Note, however, that this requires an internet connection. It is safe to skip this cell if you are not connected to the internet because we already defined the object `ngc188_center` in the cells above.

[1]If you do not know what a class method is, think of it like an alternative constructor for a `SkyCoord` object — calling `SkyCoord.from_name()` with a name gives you a new `SkyCoord` object. For more detailed background on what class methods are and when they're useful, see this page.

```
ngc188_center = SkyCoord.from_name("NGC 188")
ngc188_center
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (11.8117, 85.245)>
```

The `SkyCoord` object that we defined now has various ways of accessing the coordinate information contained within it. All `SkyCoord` objects have attributes that allow you to retrieve the coordinate component data, but the component names will change depending on the coordinate frame of the

`SkyCoord` you have. In our examples we have created a `SkyCoord` in the ICRS frame, so the component names are lower-case abbreviations of Right Ascension, `.ra`, and Declination, `.dec`:

```
ngc188_center.ra, ngc188_center.dec
```

```
(<Longitude 11.8117 deg>, <Latitude 85.245 deg>)
```

The `SkyCoord` component attributes (here `ra` and `dec`) return specialized `Quantity`-like objects that make working with angular data easier. While `Quantity` (docs) is a general class that represents numerical values and physical units of any kind, `astropy.coordinates` defines subclasses of `Quantity` that are specifically designed for working with angles, such as the `Angle` (docs) class. The `Angle` class then has additional, more specialized subclasses `Latitude` (docs) and `Longitude` (docs). These objects store angles, provide useful attributes to quickly convert to common angular units, and enable formatting the numerical values in various formats. For example, in a Jupyter notebook, these objects know how to represent themselves using LaTeX:

```
ngc188_center.ra
```

$$11^\circ 48' 42.12''$$

```
ngc188_center.dec
```

$$85^\circ 14' 42''$$

```
type(ngc188_center.ra), type(ngc188_center.dec)
```

```
(astropy.coordinates.angles.core.Longitude,
 astropy.coordinates.angles.core.Latitude)
```

With these objects, we can retrieve the coordinate components in different units using the `Quantity.to()` method:

```
(
    ngc188_center.ra.to(u.hourangle),
    ngc188_center.ra.to(u.radian),
    ngc188_center.ra.to(u.degree),
)
```

```
(<Longitude 0.78744667 hourangle>,
 <Longitude 0.20615306 rad>,
 <Longitude 11.8117 deg>)
```

Or using the shorthand attributes, which return only the component values:

```
(ngc188_center.ra.hour, ngc188_center.ra.radian, ngc188_center.ra.degree)
```

```
(np.float64(0.7874466666666669),
 np.float64(0.20615305525781422),
 np.float64(11.8117))
```

We can also format the values into strings with specified units (docs), for example:

```
ngc188_center.ra.to_string(unit=u.hourangle, sep=":", pad=True)
```

```
np.str_('00:47:14.808')
```

Querying the *Gaia* Archive to Retrieve Coordinates of Stars in NGC 188

Now that we have a `SkyCoord` object for the center of NGC 188, we can use this object with the `astroquery` package to query many different astronomical databases (see a full list of available services in the astroquery documentation). Here, we will use the `SkyCoord` object `ngc188_center` to select sources from the *Gaia* Data Release 2 catalog around the position of the center of NGC 188 to look for stars that might be members of the star cluster. To do this, we will use the `astroquery.gaia` subpackage (docs).

This requires an internet connection, but if it fails, the catalog file is included in the repository so you can load it locally (skip the next cell if you do not have an internet connection):

```
job = Gaia.cone_search_async(ngc188_center, radius=0.5 * u.deg)
ngc188_table = job.get_results()

# only keep stars brighter than G=19 magnitude
ngc188_table = ngc188_table[ngc188_table["phot_g_mean_mag"] < 19 * u.mag]
```

```
INFO: Query finished. [astroquery.utils.tap.core]
```

```
cols = [
    "source_id",
    "ra",
    "dec",
    "parallax",
    "parallax_error",
    "pmra",
    "pmdec",
    "radial_velocity",
    "phot_g_mean_mag",
    "phot_bp_mean_mag",
    "phot_rp_mean_mag",
]
ngc188_table[cols].write("gaia_results.fits", overwrite=True)
```

The above cell may not work if you do not have an internet connection, so we have included the results table along with the notebook:

```
ngc188_table = QTable.read("gaia_results.fits")
```

```
len(ngc188_table)
```

```
4870
```

The returned `astropy.table` `Table` object now contains about 5000 stars from *Gaia* DR2 around the coordinate position of the center of NGC 188. Let's now construct a `SkyCoord` object with the results table. In the *Gaia* data archive, the ICRS coordinates of a source are given as column names `"ra"` and `"dec"`:

```
ngc188_table["ra"]
```

$$[11.830258,\ 11.78143,\ 11.775831,\ \ldots,\ 13.380355,\ 6.1428143,\ 15.883725]^{\circ}$$

```
ngc188_table["dec"]
```

$$[85.243627,\ 85.247148,\ 85.241932,\ \ldots,\ 85.71469,\ 85.142685,\ 85.606562]^{\circ}$$

Note that, because the *Gaia* archive provides data tables with associated units, and we read this table using the `QTable` object (docs), the above table columns are represented as `Quantity` objects with units of degrees. Note also that these columns contain many (>5000!) coordinate values. We can pass these directly in to `SkyCoord` to get a single `SkyCoord` object to represent all of these coordinates:

```
ngc188_gaia_coords = SkyCoord(ngc188_table["ra"], ngc188_table["dec"])
ngc188_gaia_coords
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    [(11.83025781, 85.24362655), (11.78143038, 85.24714775),
     (11.77583082, 85.24193164), ..., (13.38035513, 85.71468997),
     ( 6.14281433, 85.14268532), (15.88372451, 85.60656188)]>
```

Exercises

Create a `SkyCoord` for the center of the open cluster the Pleiades (either by looking up the coordinates and passing them in, or by using the convenience method we learned about above):

```
ngc188_center = SkyCoord.from_name("NGC 188")
```

Using only a single method/function call on the `SkyCoord` object representing the center of NGC 188, print a string with the RA/Dec in the form ‘HH:MM:SS.S DD:MM:SS.S’. Check your answer against SIMBAD, which will show you sexagesimal coordinates for the object.

(Hint: `SkyCoord.to_string()` might be useful)

```
ngc188_center.to_string(style="hmsdms", sep=":", precision=1)
```

```
'00:47:14.8 +85:14:42.0'
```

Using a single method/function call on the `SkyCoord` object containing the results of our *Gaia* query, compute the angular separation between each resulting star and the coordinates of the cluster center for NGC 188.

(Hint: `SkyCoord.separation()` might be useful)

```
ngc188_gaia_coords.separation(ngc188_center)
```

$[\$0^\circ 00' 07.42471524'' \$\$0^\circ 00' 11.88878937'' \$\$0^\circ 00' 15.38406054'' \$ \ldots \$0^\circ 29' 08.31723589'' \$\$0^\circ$

More Than Just Sky Positions: Including Distance Information in `SkyCoord`

So far, we have used `SkyCoord` to represent angular sky positions (i.e., `ra` and `dec` only). It is sometimes useful to include distance information with the sky coordinates of a source, thereby specifying the full 3D position of an object. To pass in distance information, `SkyCoord` accepts the keyword argument "`distance`". So, if we knew that the distance to NGC 188 is 1.96 kpc, we could also pass in a distance (as a `Quantity` object) using this argument:

```
ngc188_center_3d = SkyCoord(12.11 * u.deg, 85.26 * u.deg, distance=1.96 * u.kpc)
```

With the table of *Gaia* data we retrieved above for stars around NGC 188, `ngc188_table`, we also have parallax measurements for each star. For a precisely-measured parallax ϖ, the distance d to a star can be obtained approximately as $d \approx 1/\varpi$. This only really works if the parallax error is small relative to the parallax (see discussion in this paper), so if we want to use these parallaxes to get distances we first have to filter out stars that have low signal-to-noise parallaxes:

```
parallax_snr = ngc188_table["parallax"] / ngc188_table["parallax_error"]
ngc188_table_3d = ngc188_table[parallax_snr > 10]
len(ngc188_table_3d)
```

```
2423
```

The above selection on `parallax_snr` keeps stars that have a ~10-sigma parallax measurement, but this is an arbitrary selection threshold that you may want to tune or remove in your own use cases. This selection removed over half of the stars in our original table, but for the remaining stars we can be confident that converting the parallax measurements to distances is mostly safe.

The default way of passing in a distance to a `SkyCoord` object, as above, is to pass in a `Quantity` with a unit of length. However, `astropy.coordinates` also provides a specialized object, `Distance`, for handling common transformations of different distance representations (docs). Among other things, this class supports passing in a parallax value:

```
Distance(parallax=1 * u.mas)
```

$$1000\ \mathrm{pc}$$

The catalog of stars we queried from *Gaia* contains parallax information in milliarcsecond units, so we can create a `Distance` object directly from these values:

```
gaia_dist = Distance(parallax=ngc188_table_3d["parallax"].filled(np.nan))
```

We can then create a `SkyCoord` object to represent the 3D positions of all of the *Gaia* stars by passing in this distance object to the `SkyCoord` initializer:

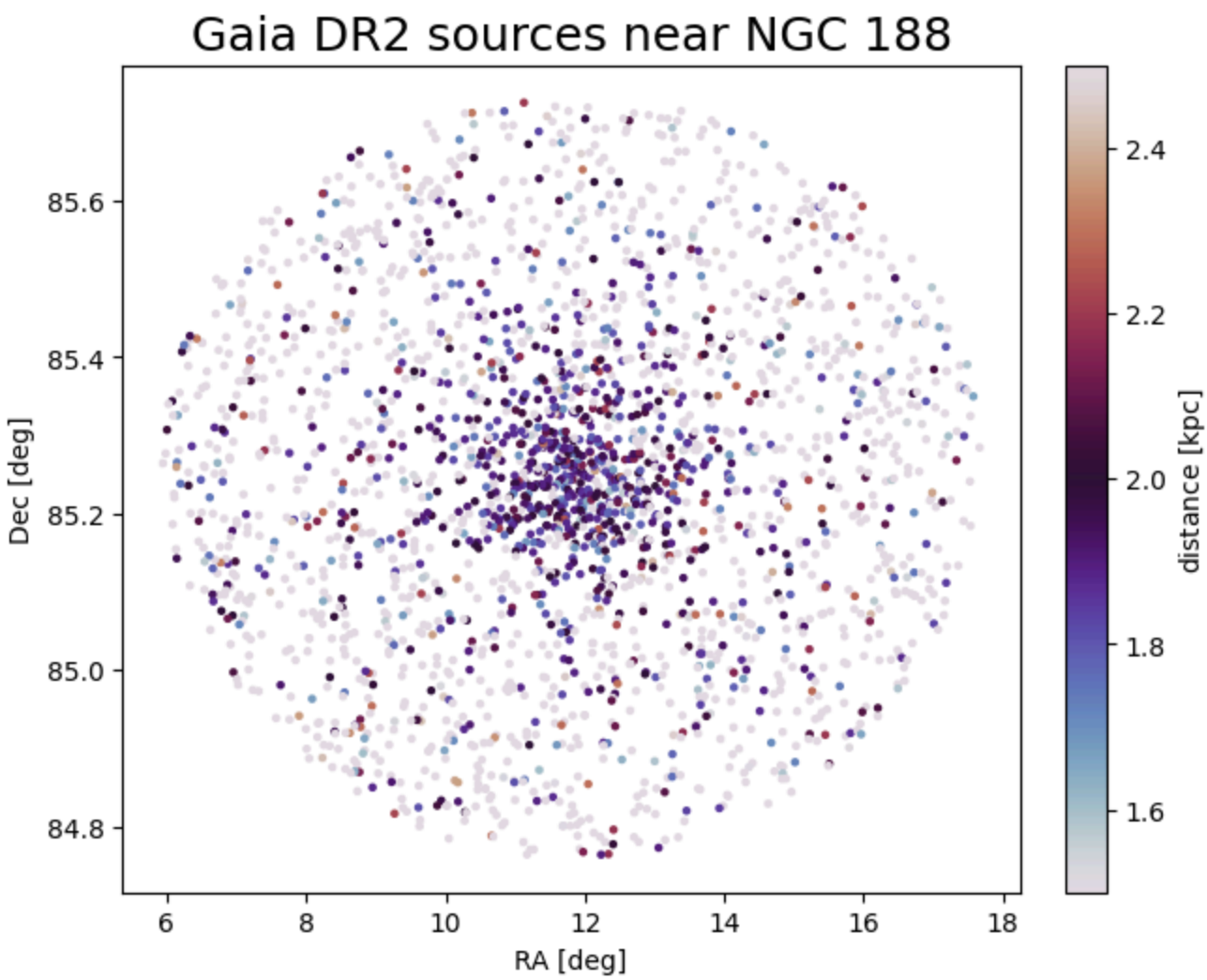
```
ngc188_coords_3d = SkyCoord(
    ra=ngc188_table_3d["ra"], dec=ngc188_table_3d["dec"], distance=gaia_dist
)
ngc188_coords_3d
```

```
<SkyCoord (ICRS): (ra, dec, distance) in (deg, deg, pc)
    [(11.83025781, 85.24362655, 2113.23414411),
     (11.77583082, 85.24193164, 1819.25788676),
     (11.79608811, 85.24947769, 1878.68206887), ...,
     (13.38035513, 85.71468997,  820.05330166),
     ( 6.14281433, 85.14268532, 1950.38491004),
     (15.88372451, 85.60656188,  356.47807826)]>
```

Let's now use `matplotlib` to plot the sky positions of all of these sources, colored by distance to emphasize the cluster stars:

```
fig, ax = plt.subplots(figsize=(6.5, 5.2), constrained_layout=True)
cs = ax.scatter(
    ngc188_coords_3d.ra.degree,
    ngc188_coords_3d.dec.degree,
    c=ngc188_coords_3d.distance.kpc,
    s=5,
    vmin=1.5,
    vmax=2.5,
    cmap="twilight",
)
cb = fig.colorbar(cs)
cb.set_label(f"distance [{u.kpc:latex_inline}]")

ax.set_xlabel("RA [deg]")
ax.set_ylabel("Dec [deg]")

ax.set_title("Gaia DR2 sources near NGC 188", fontsize=18)
```

```
Text(0.5, 1.0, 'Gaia DR2 sources near NGC 188')
```



Now that we have 3D position information for both the cluster center, and for the stars we queried from *Gaia*, we can compute the 3D separation (distance) between all of the *Gaia* sources and the cluster center:

```
sep3d = ngc188_coords_3d.separation_3d(ngc188_center_3d)
sep3d
```

$$[153.23746,\ 140.74633,\ 81.323346,\ \ldots,\ 1139.9932,\ 19.958413,\ 1603.5359]\ \mathrm{pc}$$

Exercises

Using the 3D separation values, define a boolean mask to select candidate members of the cluster. Select all stars within 50 pc of the cluster center. How many candidate members of NGC 188 do we have, based on their 3D positions?

```
ngc188_3d_mask = sep3d < 50 * u.pc
ngc188_3d_mask.sum()
```

```
np.int64(262)
```

In this tutorial, we have introduced `astropy.coordinates` as a way to store and represent astronomical sky coordinates. We used coordinate objects, via the `SkyCoord` class interface, to parse and change coordinate representations and units. We also demonstrated how to use a `SkyCoord` object with the `astroquery` package to query an astronomical database, the *Gaia* science archive. We then created a single `SkyCoord` object with the queried data that represents the sky coordinates of many objects. Finally, we introduced the concept of using `astropy.coordinates` to represent a 3D position of an object or set of objects.

Authors

Adrian Price-Whelan, Saima Siddiqui, Zihao Chen, Luthien Liu

Learning Goals

- Introduce key concepts in `astropy.coordinates`: coordinate component formats, representations, and frames
- Demonstrate how to work with coordinate representations, for example, to change from Cartesian to Cylindrical coordinates
- Introduce coordinate frame transformations and demonstrate transforming from ICRS coordinates to Galactic and Altitude-Azimuth coordinates

Keywords

coordinates, OOP

Summary

In the previous tutorial in this series, we showed how astronomical coordinates in the ICRS or equatorial coordinate system can be represented in Python using the `SkyCoord` object (docs). There are many other coordinate systems that are commonly used in astronomical research. For example, the Galactic coordinate system is often used in radio astronomy and Galactic science, the “horizontal” or altitude-azimuth frame is often used for observatory-specific observation planning, and Ecliptic coordinates are often used for solar system science or space mission footprints. All of these coordinate frames (and others!) are supported by `astropy.coordinates`. As we will see below, the `SkyCoord` object is designed to make transforming between these systems a straightforward task.

In this tutorial, we will explore how the `astropy.coordinates` package can be used to transform astronomical coordinates between different coordinate systems or frames. You may find it helpful to keep the Astropy documentation for the coordinates package open alongside this tutorial for reference or additional reading. In the text below, you may also see some links that look like (docs). These links will take you to parts of the documentation that are directly relevant to the cells from which they link.

Note: This is the 2nd tutorial in a series of tutorials about astropy.coordinates. If you are new to astropy.coordinates, you may want to start from the beginning or an earlier tutorial.

Imports

We start by importing some general packages that we will need below:

```
import matplotlib as mpl
import matplotlib.pyplot as plt

%matplotlib inline
import numpy as np

from astropy import units as u
from astropy.coordinates import SkyCoord, Galactic, EarthLocation, AltAz
import astropy.coordinates as coord
from astropy.table import QTable
from astropy.time import Time
```

Key `astropy.coordinates` Concepts: Component Formats, Representations, and Frames

Usage of the term “coordinates” is overloaded in astronomy and is often used interchangeably when referring to data formats (e.g., sexagesimal vs. decimal), representations (e.g., Cartesian vs. spherical), and frames (e.g., equatorial vs. galactic). In `astropy.coordinates`, we have tried to formalize these three concepts and have made them a core part of the way we interact with objects in this subpackage (docs). Here we will give an overview of these different concepts as we build up to demonstrating how to transform between different astronomical reference frames or systems.

Coordinate Component Formats

In our previous tutorial, we showed that it is possible to pass in coordinate component data to the `SkyCoord` initializer as strings or as `Quantity` objects in a variety of formats and units. We also saw that the coordinate components of `SkyCoord` objects can be re-formatted. For example, we can change the coordinate *format* by changing the component units, or converting the data to a string:

```
c = SkyCoord(ra=15.9932 * u.deg, dec=-10.52351344 * u.deg)
print(c.ra.hourangle)
print(c.to_string("hmsdms"))
print(c.dec.to_string(sep=":", precision=5))
```

```
1.0662133333333335
01h03m58.368s -10d31m24.648384s
-10:31:24.64838
```

See the previous tutorial [Astronomical Coordinates 1 - Getting Started](#) for more examples of this.

Coordinate Representations

In the previous tutorial, we only worked with coordinate data in spherical representations (longitude/latitude), but `astropy.coordinates` also supports other coordinate representations like Cartesian, cylindrical, etc. (docs). To retrieve the coordinate data in a different representation, we can use the `SkyCoord.represent_as()` method. This method either takes a string name of the desired representation, for example:

```
c.represent_as("cartesian")
```

```
<CartesianRepresentation (x, y, z) [dimensionless]
    (0.94512547, 0.27088898, -0.18263903)>
```

or it accepts an `astropy.coordinates` `Representation` class, such as:

```
c.represent_as(coord.CartesianRepresentation)
```

```
<CartesianRepresentation (x, y, z) [dimensionless]
    (0.94512547, 0.27088898, -0.18263903)>
```

A list of all supported representations is given in the documentation (docs), or can be identified as class names that end in `Representaton`:

```
print(
    [x for x in dir(coord) if x.endswith("Representation") and not x.startswith("Bas
)
```

```
['CartesianRepresentation', 'CylindricalRepresentation', 'GRS80GeodeticRepresentatio
```

In the `SkyCoord` object that we defined above, we only specified sky positions (i.e., no distance data), so the units of the Cartesian components that are returned above are dimensionless and are interpreted as being on the surface of the (dimensionless) unit sphere. If we instead pass in a distance to `SkyCoord` using the `distance` keyword argument, we instead get a `CartesianRepresentation` object for the 3D position with positional units. For example:

```
c2 = SkyCoord(ra=15.9932 * u.deg, dec=-10.52351344 * u.deg, distance=127.4 * u.pc)
```

```
c2.represent_as("cartesian")
```

```
<CartesianRepresentation (x, y, z) in pc
    (120.40898448, 34.5112558, -23.2682118)>
```

Or, we could represent this data with cylindrical components:

```
c2.represent_as("cylindrical")
```

```
<CylindricalRepresentation (rho, phi, z) in (pc, rad, pc)
    (125.2571368, 0.279134, -23.2682118)>
```

To summarize, using `SkyCoord.represent_as()` is a convenient way to retrieve your coordinate data in a different representation, like Cartesian or Cylindrical. You can also change (in place) the representation of a `SkyCoord` object by setting the `SkyCoord.representation_type` attribute. For example, if we create a `SkyCoord` again with a distance, the default representation type is spherical:

```
c3 = SkyCoord(ra=15.9932 * u.deg, dec=-10.52351344 * u.deg, distance=127.4 * u.pc)
print(c3.representation_type)
c3
```

```
<class 'astropy.coordinates.representation.spherical.SphericalRepresentation'>
```

```
<SkyCoord (ICRS): (ra, dec, distance) in (deg, deg, pc)
    (15.9932, -10.52351344, 127.4)>
```

We can, however, change the internal representation of the data by setting the `representation_type` attribute to a new `Representation` class:

```
c3.representation_type = coord.CylindricalRepresentation
```

This then changes the way `SkyCoord` will display the components:

```
c3
```

```
<SkyCoord (ICRS): (rho, phi, z) in (pc, deg, pc)
    (125.2571368, 15.9932, -23.2682118)>
```

Note, however, that changing the representation will also change the components that are available on a given `SkyCoord` object: Once we set the `representation_type` to cylindrical, the attributes `.ra` and `.dec` will no longer work, and we instead have to use the cylindrical component names to access the data. In this case, these are `.rho` for radius, `.phi` for azimuth, `.z` for z position:

```
c3.rho, c3.phi, c3.z
```

```
(<Quantity 125.2571368 pc>, <Angle 15.9932 deg>, <Quantity -23.2682118 pc>)
```

Transforming Between Coordinate Frames

The third key concept to keep in mind when thinking about astronomical coordinate data is the reference frame or coordinate system that the data are in. In the previous tutorial, and so far here, we have worked with the default frame assumed by `SkyCoord`: the International Celestial Reference System (ICRS; some important definitions and context about the ICRS is given here). The ICRS is the fundamental coordinate system used in most modern astronomical contexts and is generally what people mean when they refer to “equatorial” or “J2000” or “RA/Dec” coordinates (but there are some important caveats if you are working with older data). As noted above, however, there are many other coordinate systems used in different astronomical, solar, or solar system contexts.

Some other common coordinate systems are defined as a rotation away from the ICRS that is defined to make science applications easier to interpret. One example here is the Galactic coordinate system,

which is rotated with respect to the ICRS to approximately align the Galactic plane with latitude=0. As an example of the `astropy.coordinates` frame transformation machinery, we will load in a subset of a catalog of positions and distances to a set of open clusters in the Milky Way from Cantat-Gaudin et al. 2018 (Table 1 in this catalog). We have pre-selected the 474 clusters within 2 kpc of the sun and provide the catalog as a data file next to this notebook. This catalog provides sky position (columns `RAJ2000` and `DEJ2000` in the original catalog) and distance estimates (column `dmode` in the original catalog), which we have renamed in the table we provide to column names `'ra'`, `'dec'`, and `'distance'`. We will start by loading the catalog as a `QTable` using `astropy.table` (docs):

```
tbl = QTable.read("Cantat-Gaudin-open-clusters.ecsv")
```

We can now pass the coordinate components to `SkyCoord` to create a single array-valued `SkyCoord` object to represent the positions of all of the open clusters in this catalog. Note that below we will explicitly specify the coordinate frame using `frame='icrs'`: Even though this is the default frame, it is often better to be explicit so that it is clearer to someone reading the code what the coordinate system is:

```
open_cluster_c = SkyCoord(
    ra=tbl["ra"], dec=tbl["dec"], distance=tbl["distance"], frame="icrs"
)
len(open_cluster_c)
```

```
474
```

To see the first few coordinate entries, we can “slice” the array-valued coordinate object like we would a Python list or numpy array:

```
open_cluster_c[:4]
```

```
<SkyCoord (ICRS): (ra, dec, distance) in (deg, deg, pc)
    [( 51.87 , 34.981,  629.6), (288.399, 36.369,  382.2),
     (295.548, 27.366,  522.9), (298.306, 39.349, 1034.6)]>
```

Let’s now visualize the sky positions of all of these clusters, colored by their distances. To plot these in an all-sky spherical projection (e.g., aitoff) using `matplotlib`, with longitude increasing to the left as is typically done for plotting astronomical objects on the sky, we have to trick `matplotlib` a little

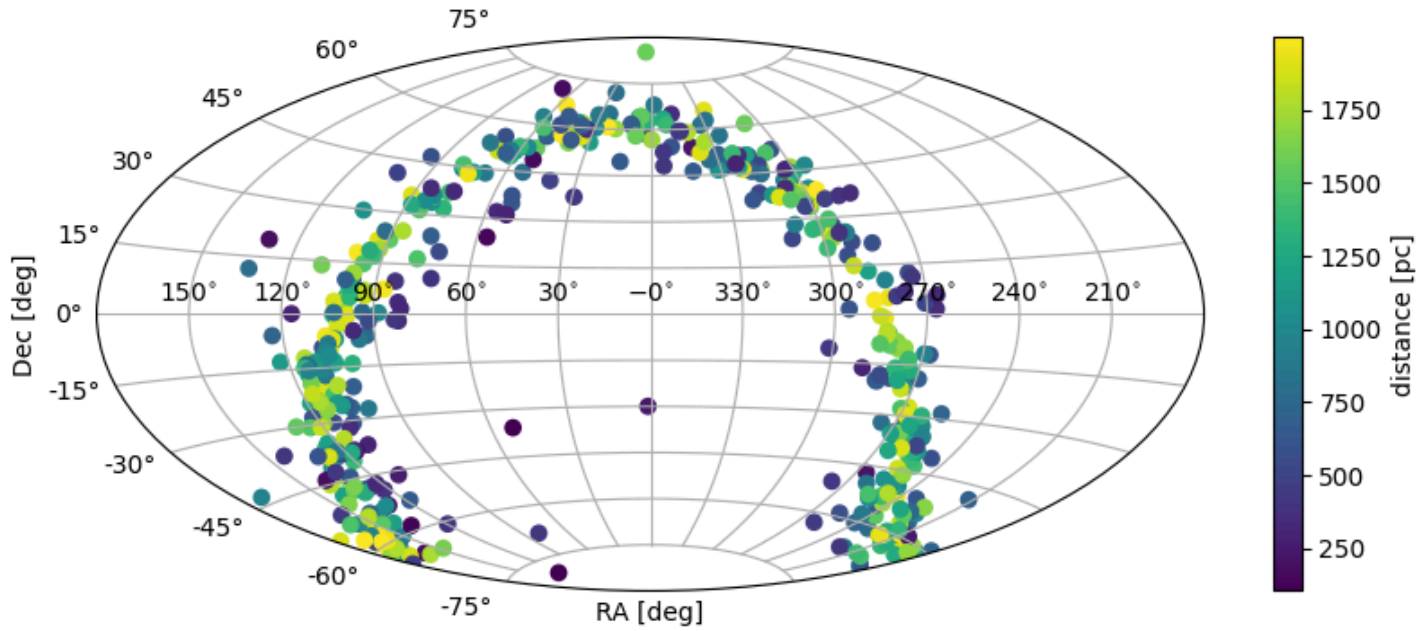
bit: We have to pass in the *negative* angle values when plotting, then reformat the tick labels to make them positive values again. We have written a short function below to handle this trick:

```
def coordinates_aitoff_plot(coords):
    fig, ax = plt.subplots(figsize=(10, 4), subplot_kw=dict(projection="aitoff"))

    sph = coords.spherical
    cs = ax.scatter(
        -sph.lon.wrap_at(180 * u.deg).radian, sph.lat.radian, c=sph.distance.value
    )

    def fmt_func(x, pos):
        val = coord.Angle(-x * u.radian).wrap_at(360 * u.deg).degree
        return f"${val:.0f}" + r"^{\circ}$"

    ticker = mpl.ticker.FuncFormatter(fmt_func)
    ax.xaxis.set_major_formatter(ticker)

    ax.grid()

    cb = fig.colorbar(cs)
    cb.set_label("distance [pc]")

    return fig, ax
```

Now we can plot the sky positions by passing our `SkyCoord` object in to this `coordinates_aitoff_plot()` plot helper function:

```
fig, ax = coordinates_aitoff_plot(open_cluster_c)
ax.set_xlabel("RA [deg]")
ax.set_ylabel("Dec [deg]")
```

```
Text(0, 0.5, 'Dec [deg]')
```



The majority of these open clusters are relatively close to the Galactic midplane, which is why they form a fairly narrow “band” on the sky in ICRS coordinates. If we transform these positions to Galactic coordinates, we would therefore expect the points to appear around the latitude $b = 0$ line.

To transform our coordinates from ICRS to Galactic (or any other coordinate system), we can use the `SkyCoord.transform_to()` method and pass in the new coordinate frame instance (in this case, `Galactic()`):

```
open_cluster_gal = open_cluster_c.transform_to(Galactic())
```

While the recommended way of transforming `SkyCoord` objects to new frames is by passing in a frame class *instance* as we demonstrated in the cell above, `SkyCoord` also supports a shorthand for transforming some frames by accessing attributes (named as the lower-case version of the new frame name):

```
open_cluster_gal = open_cluster_c.galactic
```

The transformed `SkyCoord` object now contains coordinate data in the `Galactic` coordinate frame:

```
open_cluster_gal[:4]
```

```
<SkyCoord (Galactic): (l, b, distance) in (deg, deg, pc)
    [(155.72353157, -17.76999215,  629.59997559),
     ( 68.02807936,  11.60790067,  382.20001221),
     ( 62.82445527,   2.06275608,  522.90002441),
     ( 74.37841053,   6.07393592, 1034.59997559)]>
```

Comparing this to the original `SkyCoord`, note that the names of the longitude and latitude components have changed from `ra` to `l` and from `dec` to `b`, per convention. We can therefore access the new Galactic longitude/latitude data using these new attribute names:

```
open_cluster_gal.l[:3]
```

$$[\$155^\circ 43' 24.71365331'' \$\$68^\circ 01' 41.08568525'' \$\$62^\circ 49' 28.03898608'' \$]$$

```
open_cluster_gal.b[:3]
```

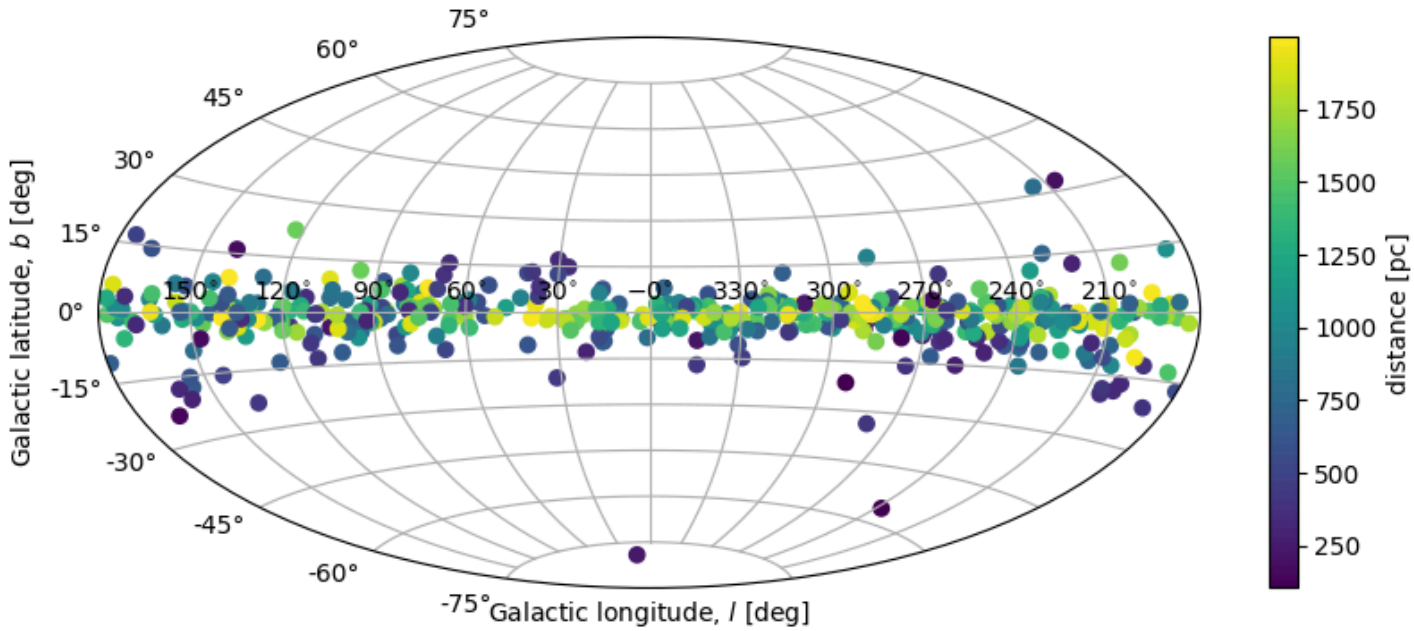
$$[\$ - 17^\circ 46' 11.97173095'' \$\$11^\circ 36' 28.44242147'' \$\$2^\circ 03' 45.92187936'' \$]$$

*Note: the ICRS coordinate component names (`.ra`, `.dec`) will therefore **not** work on this new, transformed `SkyCoord` instance, `open_cluster_gal`*

With this new `SkyCoord` object (in the Galactic frame), let's re-make a sky plot to visualize the sky positions of the open clusters in Galactic coordinates:

```
fig, ax = coordinates_aitoff_plot(open_cluster_gal)
ax.set_xlabel("Galactic longitude, $l$ [deg]")
ax.set_ylabel("Galactic latitude, $b$ [deg]")
```

```
Text(0, 0.5, 'Galactic latitude, $b$ [deg]')
```



As we hoped and expected, in the Galactic coordinate frame, the open clusters predominantly appear at low galactic latitudes!

Transforming to More Complex Coordinate Frames: Computing the Altitude of a Target at an Observatory

To determine whether a target is observable from a given observatory on Earth or to find out what targets are observable from a city or place on Earth at some time, we sometimes need to convert a coordinate or set of coordinates to a frame that is local to an on-earth observer. The most common choice for such a frame is “horizontal” or “Altitude-Azimuth” coordinates. In this frame, the sky coordinates of a source can be specified as an altitude from the horizon and an azimuth angle at a specified time. This coordinate frame is supported in `astropy.coordinates` through the `AltAz` coordinate frame.

The `AltAz` frame is different from the previously-demonstrated `Galactic` frame in that it requires additional metadata to define the frame instance. Since the `Galactic` frame is close to being a 3D rotation away from the ICRS frame, and that rotation matrix is fixed, we could transform to `Galactic` by instantiating the class with no arguments (see the example above where we used

`.transform_to(Galactic())`). In order to specify an instance of the `AltAz` frame, we have to (at minimum) pass in (1) a location on Earth, and (2) the time (or times) we are requesting the frame at.

In `astropy.coordinates`, we specify locations on Earth with the `EarthLocation` class (docs). If we know the Earth longitude and latitude of our site, we can use these to create an instance of `EarthLocation` directly:

```
demo_loc = EarthLocation.from_geodetic(lon=-74.32834 * u.deg, lat=43.05885 * u.deg)
```

The `EarthLocation` class also provides handy short-hands for retrieving an instance for a given street address (by querying the OpenStreetMap web API):

```
demo_loc = EarthLocation.of_address("162 Fifth Ave, New York, NY 10010")
```

Or for an astronomical observatory (use `EarthLocation.get_site_names()` to see a list of all available sites). For example, to retrieve an `EarthLocation` instance for the position of Kitt Peak National Observatory (in AZ, USA):

```
observing_location = EarthLocation.of_site("Kitt Peak")
```

We will use Kitt Peak as our site.

As an example, we will now compute the altitude of a few of the open clusters from our catalog above over the course of a night. We have an object to represent our location on Earth, so now we need to create a set of times to compute the `AltAz` frame for. `AltAz` expects time information to be passed in as an `astropy.time.Time` object (docs; which can contain an array of times). Let's pretend we have an observing run coming up on Dec 18, 2020, and we would like to compute the altitude/azimuth coordinates for our open clusters over that whole night.

```
# 1AM UTC = 6PM local time (AZ mountain time), roughly the start of a night
observing_date = Time("2020-12-18 1:00")

# Compute the alt/az over a 14 hour period, starting at 6PM local time,
# with 256 equally spaced time points:
time_grid = observing_date + np.linspace(0, 14, 256) * u.hour
```

Now we use our location, `observing_location`, and this grid of times, `time_grid`, to create an `AltAz` frame object.

Note: This frame accepts even more parameters about the atmosphere, which can be used to correct for atmospheric refraction. But here we leave those additional parameters set to their defaults, which ignores refraction.

```
altaz = AltAz(location=observing_location, obstime=time_grid)
```

Now we can transform the ICRS `SkyCoord` positions of the open clusters to `AltAz` to get the location of each of the clusters in the sky over Kitt Peak over a night. Let's first do this only for the first open cluster in the catalog we loaded:

```
oc_altaz = open_cluster_c[0].transform_to(altaz)
oc_altaz
```

```
<SkyCoord (AltAz: obstime=['2020-12-18 01:00:00.000' '2020-12-18 01:03:17.647'
 '2020-12-18 01:06:35.294' '2020-12-18 01:09:52.941'
 '2020-12-18 01:13:10.588' '2020-12-18 01:16:28.235'
 '2020-12-18 01:19:45.882' '2020-12-18 01:23:03.529'
 '2020-12-18 01:26:21.176' '2020-12-18 01:29:38.824'
 '2020-12-18 01:32:56.471' '2020-12-18 01:36:14.118'
 '2020-12-18 01:39:31.765' '2020-12-18 01:42:49.412'
 '2020-12-18 01:46:07.059' '2020-12-18 01:49:24.706'
 '2020-12-18 01:52:42.353' '2020-12-18 01:56:00.000'
 '2020-12-18 01:59:17.647' '2020-12-18 02:02:35.294'
 '2020-12-18 02:05:52.941' '2020-12-18 02:09:10.588'
 '2020-12-18 02:12:28.235' '2020-12-18 02:15:45.882'
 '2020-12-18 02:19:03.529' '2020-12-18 02:22:21.176'
 '2020-12-18 02:25:38.824' '2020-12-18 02:28:56.471'
 '2020-12-18 02:32:14.118' '2020-12-18 02:35:31.765'
 '2020-12-18 02:38:49.412' '2020-12-18 02:42:07.059'
 '2020-12-18 02:45:24.706' '2020-12-18 02:48:42.353'
 '2020-12-18 02:52:00.000' '2020-12-18 02:55:17.647'
 '2020-12-18 02:58:35.294' '2020-12-18 03:01:52.941'
 '2020-12-18 03:05:10.588' '2020-12-18 03:08:28.235'
 '2020-12-18 03:11:45.882' '2020-12-18 03:15:03.529'
 '2020-12-18 03:18:21.176' '2020-12-18 03:21:38.824'
 '2020-12-18 03:24:56.471' '2020-12-18 03:28:14.118'
 '2020-12-18 03:31:31.765' '2020-12-18 03:34:49.412'
 '2020-12-18 03:38:07.059' '2020-12-18 03:41:24.706'
 '2020-12-18 03:44:42.353' '2020-12-18 03:48:00.000'
 '2020-12-18 03:51:17.647' '2020-12-18 03:54:35.294'
 '2020-12-18 03:57:52.941' '2020-12-18 04:01:10.588'
 '2020-12-18 04:04:28.235' '2020-12-18 04:07:45.882'
 '2020-12-18 04:11:03.529' '2020-12-18 04:14:21.176'
 '2020-12-18 04:17:38.824' '2020-12-18 04:20:56.471'
 '2020-12-18 04:24:14.118' '2020-12-18 04:27:31.765'
 '2020-12-18 04:30:49.412' '2020-12-18 04:34:07.059'
 '2020-12-18 04:37:24.706' '2020-12-18 04:40:42.353'
 '2020-12-18 04:44:00.000' '2020-12-18 04:47:17.647'
 '2020-12-18 04:50:35.294' '2020-12-18 04:53:52.941'
 '2020-12-18 04:57:10.588' '2020-12-18 05:00:28.235'
 '2020-12-18 05:03:45.882' '2020-12-18 05:07:03.529'
 '2020-12-18 05:10:21.176' '2020-12-18 05:13:38.824'
 '2020-12-18 05:16:56.471' '2020-12-18 05:20:14.118'
 '2020-12-18 05:23:31.765' '2020-12-18 05:26:49.412'
 '2020-12-18 05:30:07.059' '2020-12-18 05:33:24.706'
 '2020-12-18 05:36:42.353' '2020-12-18 05:40:00.000'
 '2020-12-18 05:43:17.647' '2020-12-18 05:46:35.294'
 '2020-12-18 05:49:52.941' '2020-12-18 05:53:10.588'
 '2020-12-18 05:56:28.235' '2020-12-18 05:59:45.882'
 '2020-12-18 06:03:03.529' '2020-12-18 06:06:21.176'
 '2020-12-18 06:09:38.824' '2020-12-18 06:12:56.471'
 '2020-12-18 06:16:14.118' '2020-12-18 06:19:31.765'
 '2020-12-18 06:22:49.412' '2020-12-18 06:26:07.059'
 '2020-12-18 06:29:24.706' '2020-12-18 06:32:42.353'
 '2020-12-18 06:36:00.000' '2020-12-18 06:39:17.647'
 '2020-12-18 06:42:35.294' '2020-12-18 06:45:52.941'
```

'2020-12-18 06:49:10.588' '2020-12-18 06:52:28.235'
'2020-12-18 06:55:45.882' '2020-12-18 06:59:03.529'
'2020-12-18 07:02:21.176' '2020-12-18 07:05:38.824'
'2020-12-18 07:08:56.471' '2020-12-18 07:12:14.118'
'2020-12-18 07:15:31.765' '2020-12-18 07:18:49.412'
'2020-12-18 07:22:07.059' '2020-12-18 07:25:24.706'
'2020-12-18 07:28:42.353' '2020-12-18 07:32:00.000'
'2020-12-18 07:35:17.647' '2020-12-18 07:38:35.294'
'2020-12-18 07:41:52.941' '2020-12-18 07:45:10.588'
'2020-12-18 07:48:28.235' '2020-12-18 07:51:45.882'
'2020-12-18 07:55:03.529' '2020-12-18 07:58:21.176'
'2020-12-18 08:01:38.824' '2020-12-18 08:04:56.471'
'2020-12-18 08:08:14.118' '2020-12-18 08:11:31.765'
'2020-12-18 08:14:49.412' '2020-12-18 08:18:07.059'
'2020-12-18 08:21:24.706' '2020-12-18 08:24:42.353'
'2020-12-18 08:28:00.000' '2020-12-18 08:31:17.647'
'2020-12-18 08:34:35.294' '2020-12-18 08:37:52.941'
'2020-12-18 08:41:10.588' '2020-12-18 08:44:28.235'
'2020-12-18 08:47:45.882' '2020-12-18 08:51:03.529'
'2020-12-18 08:54:21.176' '2020-12-18 08:57:38.824'
'2020-12-18 09:00:56.471' '2020-12-18 09:04:14.118'
'2020-12-18 09:07:31.765' '2020-12-18 09:10:49.412'
'2020-12-18 09:14:07.059' '2020-12-18 09:17:24.706'
'2020-12-18 09:20:42.353' '2020-12-18 09:24:00.000'
'2020-12-18 09:27:17.647' '2020-12-18 09:30:35.294'
'2020-12-18 09:33:52.941' '2020-12-18 09:37:10.588'
'2020-12-18 09:40:28.235' '2020-12-18 09:43:45.882'
'2020-12-18 09:47:03.529' '2020-12-18 09:50:21.176'
'2020-12-18 09:53:38.824' '2020-12-18 09:56:56.471'
'2020-12-18 10:00:14.118' '2020-12-18 10:03:31.765'
'2020-12-18 10:06:49.412' '2020-12-18 10:10:07.059'
'2020-12-18 10:13:24.706' '2020-12-18 10:16:42.353'
'2020-12-18 10:20:00.000' '2020-12-18 10:23:17.647'
'2020-12-18 10:26:35.294' '2020-12-18 10:29:52.941'
'2020-12-18 10:33:10.588' '2020-12-18 10:36:28.235'
'2020-12-18 10:39:45.882' '2020-12-18 10:43:03.529'
'2020-12-18 10:46:21.176' '2020-12-18 10:49:38.824'
'2020-12-18 10:52:56.471' '2020-12-18 10:56:14.118'
'2020-12-18 10:59:31.765' '2020-12-18 11:02:49.412'
'2020-12-18 11:06:07.059' '2020-12-18 11:09:24.706'
'2020-12-18 11:12:42.353' '2020-12-18 11:16:00.000'
'2020-12-18 11:19:17.647' '2020-12-18 11:22:35.294'
'2020-12-18 11:25:52.941' '2020-12-18 11:29:10.588'
'2020-12-18 11:32:28.235' '2020-12-18 11:35:45.882'
'2020-12-18 11:39:03.529' '2020-12-18 11:42:21.176'
'2020-12-18 11:45:38.824' '2020-12-18 11:48:56.471'
'2020-12-18 11:52:14.118' '2020-12-18 11:55:31.765'
'2020-12-18 11:58:49.412' '2020-12-18 12:02:07.059'
'2020-12-18 12:05:24.706' '2020-12-18 12:08:42.353'
'2020-12-18 12:12:00.000' '2020-12-18 12:15:17.647'
'2020-12-18 12:18:35.294' '2020-12-18 12:21:52.941'
'2020-12-18 12:25:10.588' '2020-12-18 12:28:28.235'
'2020-12-18 12:31:45.882' '2020-12-18 12:35:03.529'
'2020-12-18 12:38:21.176' '2020-12-18 12:41:38.824'

```
'2020-12-18 12:44:56.471' '2020-12-18 12:48:14.118'
'2020-12-18 12:51:31.765' '2020-12-18 12:54:49.412'
'2020-12-18 12:58:07.059' '2020-12-18 13:01:24.706'
'2020-12-18 13:04:42.353' '2020-12-18 13:08:00.000'
'2020-12-18 13:11:17.647' '2020-12-18 13:14:35.294'
'2020-12-18 13:17:52.941' '2020-12-18 13:21:10.588'
'2020-12-18 13:24:28.235' '2020-12-18 13:27:45.882'
'2020-12-18 13:31:03.529' '2020-12-18 13:34:21.176'
'2020-12-18 13:37:38.824' '2020-12-18 13:40:56.471'
'2020-12-18 13:44:14.118' '2020-12-18 13:47:31.765'
'2020-12-18 13:50:49.412' '2020-12-18 13:54:07.059'
'2020-12-18 13:57:24.706' '2020-12-18 14:00:42.353'
'2020-12-18 14:04:00.000' '2020-12-18 14:07:17.647'
'2020-12-18 14:10:35.294' '2020-12-18 14:13:52.941'
'2020-12-18 14:17:10.588' '2020-12-18 14:20:28.235'
'2020-12-18 14:23:45.882' '2020-12-18 14:27:03.529'
'2020-12-18 14:30:21.176' '2020-12-18 14:33:38.824'
'2020-12-18 14:36:56.471' '2020-12-18 14:40:14.118'
'2020-12-18 14:43:31.765' '2020-12-18 14:46:49.412'
'2020-12-18 14:50:07.059' '2020-12-18 14:53:24.706'
'2020-12-18 14:56:42.353' '2020-12-18 15:00:00.000'], location=(-1994502.604306139,
   [( 68.6453125 ,  3.92957964e+01, 629.59997152),
    ( 68.87232312,  3.99487849e+01, 629.59997152),
    ( 69.09658831,  4.06027679e+01, 629.59997152),
    ( 69.31808233,  4.12577232e+01, 629.59997152),
    ( 69.53677613,  4.19136292e+01, 629.59997152),
    ( 69.75263715,  4.25704639e+01, 629.59997152),
    ( 69.9656291 ,  4.32282062e+01, 629.59997152),
    ( 70.17571173,  4.38868345e+01, 629.59997152),
    ( 70.38284052,  4.45463280e+01, 629.59997152),
    ( 70.58696641,  4.52066657e+01, 629.59997152),
    ( 70.7880355 ,  4.58678268e+01, 629.59997152),
    ( 70.98598866,  4.65297907e+01, 629.59997152),
    ( 71.18076119,  4.71925370e+01, 629.59997152),
    ( 71.37228235,  4.78560452e+01, 629.59997152),
    ( 71.56047493,  4.85202949e+01, 629.59997152),
    ( 71.74525474,  4.91852660e+01, 629.59997152),
    ( 71.92653004,  4.98509381e+01, 629.59997152),
    ( 72.10420091,  5.05172912e+01, 629.59997152),
    ( 72.2781586 ,  5.11843049e+01, 629.59997152),
    ( 72.44828473,  5.18519591e+01, 629.59997152),
    ( 72.61445048,  5.25202334e+01, 629.59997152),
    ( 72.77651565,  5.31891073e+01, 629.59997152),
    ( 72.9343276 ,  5.38585605e+01, 629.59997152),
    ( 73.08772013,  5.45285720e+01, 629.59997152),
    ( 73.23651217,  5.51991212e+01, 629.59997152),
    ( 73.38050632,  5.58701867e+01, 629.59997152),
    ( 73.51948725,  5.65417471e+01, 629.59997152),
    ( 73.65321987,  5.72137807e+01, 629.59997152),
    ( 73.78144728,  5.78862651e+01, 629.59997152),
    ( 73.90388843,  5.85591777e+01, 629.59997152),
    ( 74.02023555,  5.92324952e+01, 629.59997152),
    ( 74.13015113,  5.99061937e+01, 629.59997152),
    ( 74.23326456,  6.05802484e+01, 629.59997152),
```

```
( 74.32916826,  6.12546340e+01, 629.59997152),
( 74.4174133 ,  6.19293238e+01, 629.59997152),
( 74.4975043 ,  6.26042903e+01, 629.59997152),
( 74.56889364,  6.32795045e+01, 629.59997152),
( 74.63097474,  6.39549360e+01, 629.59997152),
( 74.68307425,  6.46305528e+01, 629.59997152),
( 74.72444303,  6.53063208e+01, 629.59997152),
( 74.75424559,  6.59822037e+01, 629.59997152),
( 74.77154774,  6.66581626e+01, 629.59997152),
( 74.775302  ,  6.73341557e+01, 629.59997152),
( 74.7643305 ,  6.80101374e+01, 629.59997152),
( 74.73730454,  6.86860586e+01, 629.59997152),
( 74.69272033,  6.93618648e+01, 629.59997152),
( 74.62886986,  7.00374965e+01, 629.59997152),
( 74.54380585,  7.07128874e+01, 629.59997152),
( 74.43529924,  7.13879634e+01, 629.59997152),
( 74.30078745,  7.20626413e+01, 629.59997152),
( 74.13731089,  7.27368270e+01, 629.59997152),
( 73.94143462,  7.34104128e+01, 629.59997152),
( 73.70915091,  7.40832754e+01, 629.59997153),
( 73.43575731,  7.47552717e+01, 629.59997153),
( 73.11570266,  7.54262347e+01, 629.59997153),
( 72.74239115,  7.60959677e+01, 629.59997153),
( 72.30793074,  7.67642369e+01, 629.59997153),
( 71.80280707,  7.74307620e+01, 629.59997153),
( 71.21545687,  7.80952030e+01, 629.59997153),
( 70.53170387,  7.87571438e+01, 629.59997153),
( 69.73400548,  7.94160683e+01, 629.59997153),
( 68.80043555,  8.00713294e+01, 629.59997153),
( 67.7032968 ,  8.07221043e+01, 629.59997153),
( 66.4072092 ,  8.13673318e+01, 629.59997153),
( 64.86645553,  8.20056224e+01, 629.59997153),
( 63.02127816,  8.26351262e+01, 629.59997153),
( 60.79272411,  8.32533373e+01, 629.59997153),
( 58.07557898,  8.38568002e+01, 629.59997153),
( 54.72909461,  8.44406656e+01, 629.59997153),
( 50.56612067,  8.49980225e+01, 629.59997153),
( 45.34425335,  8.55189269e+01, 629.59997153),
( 38.77063621,  8.59891116e+01, 629.59997153),
( 30.54878284,  8.63886760e+01, 629.59997153),
( 20.51471432,  8.66919017e+01, 629.59997153),
(  8.87923794,  8.68706359e+01, 629.59997153),
(356.42132249,  8.69032357e+01, 629.59997153),
(344.3056594 ,  8.67852503e+01, 629.59997153),
(333.52847465,  8.65320218e+01, 629.59997153),
(324.53491904,  8.61702986e+01, 629.59997153),
(317.28825519,  8.57275688e+01, 629.59997153),
(311.52521889,  8.52263419e+01, 629.59997153),
(306.94165647,  8.46831629e+01, 629.59997153),
(303.27141217,  8.41096560e+01, 629.59997153),
(300.30435917,  8.35138923e+01, 629.59997153),
(297.8812238 ,  8.29015066e+01, 629.59997153),
(295.88283094,  8.22764861e+01, 629.59997153),
(294.21996954,  8.16416999e+01, 629.59997153),
```

```
(292.8253914 ,  8.09992504e+01, 629.59997153),
(291.64789255,  8.03507049e+01, 629.59997153),
(290.64804286,  7.96972524e+01, 629.59997153),
(289.79512546,  7.90398085e+01, 629.59997153),
(289.06493832,  7.83790896e+01, 629.59997153),
(288.43820429,  7.77156631e+01, 629.59997153),
(287.8994107 ,  7.70499848e+01, 629.59997153),
(287.43595363,  7.63824248e+01, 629.59997153),
(287.0375003 ,  7.57132874e+01, 629.59997153),
(286.69550879,  7.50428249e+01, 629.59997153),
(286.40286268,  7.43712488e+01, 629.59997153),
(286.15359017,  7.36987380e+01, 629.59997153),
(285.9426463 ,  7.30254454e+01, 629.59997153),
(285.76574234,  7.23515022e+01, 629.59997153),
(285.61921119,  7.16770224e+01, 629.59997153),
(285.49990024,  7.10021056e+01, 629.59997153),
(285.40508539,  7.03268395e+01, 629.59997153),
(285.33240167,  6.96513019e+01, 629.59997153),
(285.27978679,  6.89755622e+01, 629.59997153),
(285.24543494,  6.82996827e+01, 629.59997153),
(285.22775872,  6.76237200e+01, 629.59997153),
(285.22535766,  6.69477254e+01, 629.59997153),
(285.23699199,  6.62717459e+01, 629.59997153),
(285.26156065,  6.55958249e+01, 629.59997153),
(285.29808283,  6.49200024e+01, 629.59997153),
(285.34568241,  6.42443159e+01, 629.59997153),
(285.40357463,  6.35688002e+01, 629.59997153),
(285.47105488,  6.28934884e+01, 629.59997153),
(285.54748899,  6.22184115e+01, 629.59997153),
(285.63230496,  6.15435989e+01, 629.59997153),
(285.72498581,  6.08690790e+01, 629.59997153),
(285.82506336,  6.01948786e+01, 629.59997153),
(285.9321129 ,  5.95210236e+01, 629.59997153),
(286.04574847,  5.88475391e+01, 629.59997153),
(286.16561884,  5.81744494e+01, 629.59997153),
(286.29140388,  5.75017779e+01, 629.59997153),
(286.42281143,  5.68295476e+01, 629.59997153),
(286.55957455,  5.61577810e+01, 629.59997153),
(286.70144908,  5.54864999e+01, 629.59997153),
(286.84821144,  5.48157261e+01, 629.59997153),
(286.99965678,  5.41454808e+01, 629.59997153),
(287.15559721,  5.34757850e+01, 629.59997153),
(287.31586033,  5.28066595e+01, 629.59997153),
(287.48028784,  5.21381250e+01, 629.59997153),
(287.64873435,  5.14702017e+01, 629.59997153),
(287.82106628,  5.08029102e+01, 629.59997153),
(287.99716089,  5.01362707e+01, 629.59997153),
(288.1769054 ,  4.94703035e+01, 629.59997153),
(288.3601962 ,  4.88050286e+01, 629.59997153),
(288.54693811,  4.81404664e+01, 629.59997153),
(288.73704379,  4.74766371e+01, 629.59997153),
(288.9304331 ,  4.68135610e+01, 629.59997153),
(289.1270326 ,  4.61512585e+01, 629.59997153),
(289.32677504,  4.54897499e+01, 629.59997153),
```

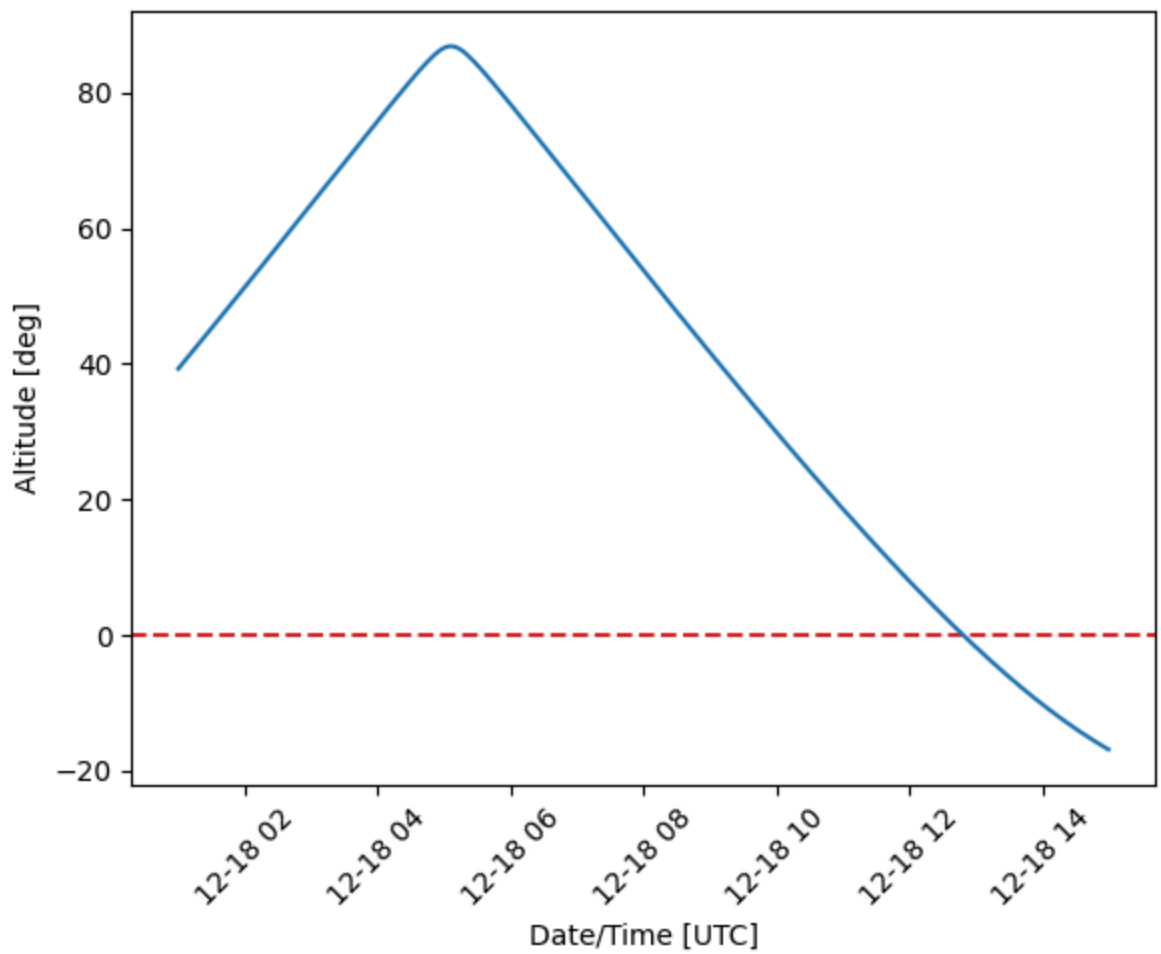
```
(289.52959897,  4.48290560e+01, 629.59997153),
(289.73544828,  4.41691973e+01, 629.59997153),
(289.94427189,  4.35101947e+01, 629.59997153),
(290.15602336,  4.28520693e+01, 629.59997153),
(290.37066066,  4.21948422e+01, 629.59997153),
(290.58814581,  4.15385347e+01, 629.59997153),
(290.80844471,  4.08831684e+01, 629.59997153),
(291.03152682,  4.02287651e+01, 629.59997153),
(291.257365  ,  3.95753469e+01, 629.59997153),
(291.48593532,  3.89229359e+01, 629.59997153),
(291.71721684,  3.82715547e+01, 629.59997153),
(291.95119145,  3.76212260e+01, 629.59997153),
(292.18784374,  3.69719729e+01, 629.59997153),
(292.42716085,  3.63238188e+01, 629.59997153),
(292.66913231,  3.56767873e+01, 629.59997153),
(292.91374996,  3.50309024e+01, 629.59997153),
(293.16100781,  3.43861884e+01, 629.59997153),
(293.41090195,  3.37426698e+01, 629.59997153),
(293.66343043,  3.31003717e+01, 629.59997153),
(293.9185932 ,  3.24593194e+01, 629.59997153),
(294.17639201,  3.18195387e+01, 629.59997153),
(294.43683035,  3.11810555e+01, 629.59997153),
(294.69991333,  3.05438965e+01, 629.59997153),
(294.96564769,  2.99080885e+01, 629.59997154),
(295.23404165,  2.92736588e+01, 629.59997154),
(295.50510492,  2.86406351e+01, 629.59997154),
(295.77884863,  2.80090457e+01, 629.59997154),
(296.05528525,  2.73789192e+01, 629.59997154),
(296.33442856,  2.67502846e+01, 629.59997154),
(296.61629362,  2.61231716e+01, 629.59997154),
(296.9008967 ,  2.54976102e+01, 629.59997154),
(297.18825527,  2.48736310e+01, 629.59997154),
(297.47838794,  2.42512649e+01, 629.59997154),
(297.77131441,  2.36305437e+01, 629.59997154),
(298.06705548,  2.30114994e+01, 629.59997154),
(298.36563299,  2.23941647e+01, 629.59997154),
(298.66706979,  2.17785729e+01, 629.59997154),
(298.97138971,  2.11647577e+01, 629.59997154),
(299.27861753,  2.05527535e+01, 629.59997154),
(299.58877898,  1.99425953e+01, 629.59997154),
(299.90190067,  1.93343187e+01, 629.59997154),
(300.2180101 ,  1.87279598e+01, 629.59997154),
(300.53713562,  1.81235557e+01, 629.59997154),
(300.8593064 ,  1.75211436e+01, 629.59997154),
(301.18455243,  1.69207618e+01, 629.59997154),
(301.51290446,  1.63224491e+01, 629.59997154),
(301.84439401,  1.57262451e+01, 629.59997154),
(302.17905333,  1.51321898e+01, 629.59997154),
(302.51691539,  1.45403243e+01, 629.59997154),
(302.85801384,  1.39506902e+01, 629.59997154),
(303.202383  ,  1.33633298e+01, 629.59997154),
(303.55005782,  1.27782864e+01, 629.59997154),
(303.90107389,  1.21956037e+01, 629.59997154),
(304.25546736,  1.16153265e+01, 629.59997154),
```

```
(304.61327499,  1.10375003e+01, 629.59997154),
(304.97453405,  1.04621712e+01, 629.59997154),
(305.33928233,  9.88938648e+00, 629.59997154),
(305.70755814,  9.31919396e+00, 629.59997154),
(306.07940021,  8.75164240e+00, 629.59997154),
(306.45484773,  8.18678143e+00, 629.59997154),
(306.83394028,  7.62466151e+00, 629.59997154),
(307.21671783,  7.06533401e+00, 629.59997154),
(307.60322067,  6.50885117e+00, 629.59997154),
(307.99348941,  5.95526613e+00, 629.59997154),
(308.3875649 ,  5.40463297e+00, 629.59997154),
(308.78548828,  4.85700665e+00, 629.59997154),
(309.18730082,  4.31244310e+00, 629.59997154),
(309.593044  ,  3.77099917e+00, 629.59997154),
(310.00275939,  3.23273269e+00, 629.59997154),
(310.41648862,  2.69770243e+00, 629.59997154),
(310.83427336,  2.16596814e+00, 629.59997154),
(311.25615526,  1.63759053e+00, 629.59997154),
(311.68217588,  1.11263133e+00, 629.59997154),
(312.11237667,  5.91153243e-01, 629.59997154),
(312.5467989 ,  7.32199602e-02, 629.59997154),
(312.98548358, -4.41103810e-01, 629.59997154),
(313.42847146, -9.51752358e-01, 629.59997154),
(313.87580291, -1.45865897e+00, 629.59997154),
(314.32751788, -1.96175590e+00, 629.59997154),
(314.78365585, -2.46097443e+00, 629.59997154),
(315.24425573, -2.95624480e+00, 629.59997154),
(315.70935579, -3.44749624e+00, 629.59997154),
(316.17899362, -3.93465700e+00, 629.59997154),
(316.65320601, -4.41765429e+00, 629.59997154),
(317.13202889, -4.89641437e+00, 629.59997154),
(317.61549724, -5.37086248e+00, 629.59997154),
(318.10364503, -5.84092289e+00, 629.59997154),
(318.59650507, -6.30651890e+00, 629.59997154),
(319.094109  , -6.76757286e+00, 629.59997154),
(319.59648711, -7.22400619e+00, 629.59997154),
(320.10366831, -7.67573936e+00, 629.59997154),
(320.61567999, -8.12269198e+00, 629.59997154),
(321.13254794, -8.56478273e+00, 629.59997154),
(321.65429623, -9.00192947e+00, 629.59997154),
(322.1809471 , -9.43404919e+00, 629.59997154),
(322.71252086, -9.86105813e+00, 629.59997154),
(323.24903577, -1.02828717e+01, 629.59997154),
(323.79050794, -1.06994046e+01, 629.59997154),
(324.33695118, -1.11105708e+01, 629.59997154),
(324.88837693, -1.15162837e+01, 629.59997154),
(325.44479408, -1.19164560e+01, 629.59997154),
(326.00620892, -1.23109997e+01, 629.59997154),
(326.57262496, -1.26998265e+01, 629.59997154),
(327.14404281, -1.30828476e+01, 629.59997154),
(327.72046011, -1.34599736e+01, 629.59997154),
(328.30187136, -1.38311149e+01, 629.59997154),
(328.88826781, -1.41961815e+01, 629.59997154),
(329.47963733, -1.45550832e+01, 629.59997154),
```

```
(330.07596431, -1.49077297e+01, 629.59997154),
(330.67722954, -1.52540304e+01, 629.59997154),
(331.28341007, -1.55938949e+01, 629.59997154),
(331.89447912, -1.59272327e+01, 629.59997154),
(332.51040596, -1.62539535e+01, 629.59997154),
(333.13115581, -1.65739671e+01, 629.59997154),
(333.75668973, -1.68871838e+01, 629.59997154)]>
```

There is a lot of information in the representation of our transformed `SkyCoord`, but note that the frame of the new object is now correctly noted as `AltAz`, as in `<SkyCoord (AltAz: ...`. Like transforming to `Galactic` coordinates above, the new `SkyCoord` object now contains the data in a new representation, so the ICRS component names `.ra` and `.dec` will not work on this new object. Instead, the data (the altitude and azimuth as a function of time) can be accessed with the `.alt` and `.az` component names. For example, let's plot the altitude of this open cluster over the course of the night:

```
plt.figure(figsize=(6, 5))
plt.plot(time_grid.datetime, oc_altaz.alt.degree, marker="")
plt.axhline(0, zorder=-10, linestyle="--", color="tab:red")
plt.xlabel("Date/Time [UTC]")
plt.ylabel("Altitude [deg]")
plt.setp(plt.gca().xaxis.get_majorticklabels(), rotation=45)
plt.tight_layout()
```



Here we can see that this open cluster reaches a high altitude above the horizon from Kitt Peak, and so it looks like it would be observable from this site. The above curve only shows the altitude trajectory for the first open cluster in our catalog, but we would like to compute the equivalent for *all* of the open clusters in the catalog. To do this, we have to make use of a concept that is used heavily in Numpy: [array broadcasting](). We have 474 open clusters and we want to evaluate the `AltAz` coordinates of these clusters at 256 different times.

```
len(open_cluster_c), len(altaz.obstime)
```

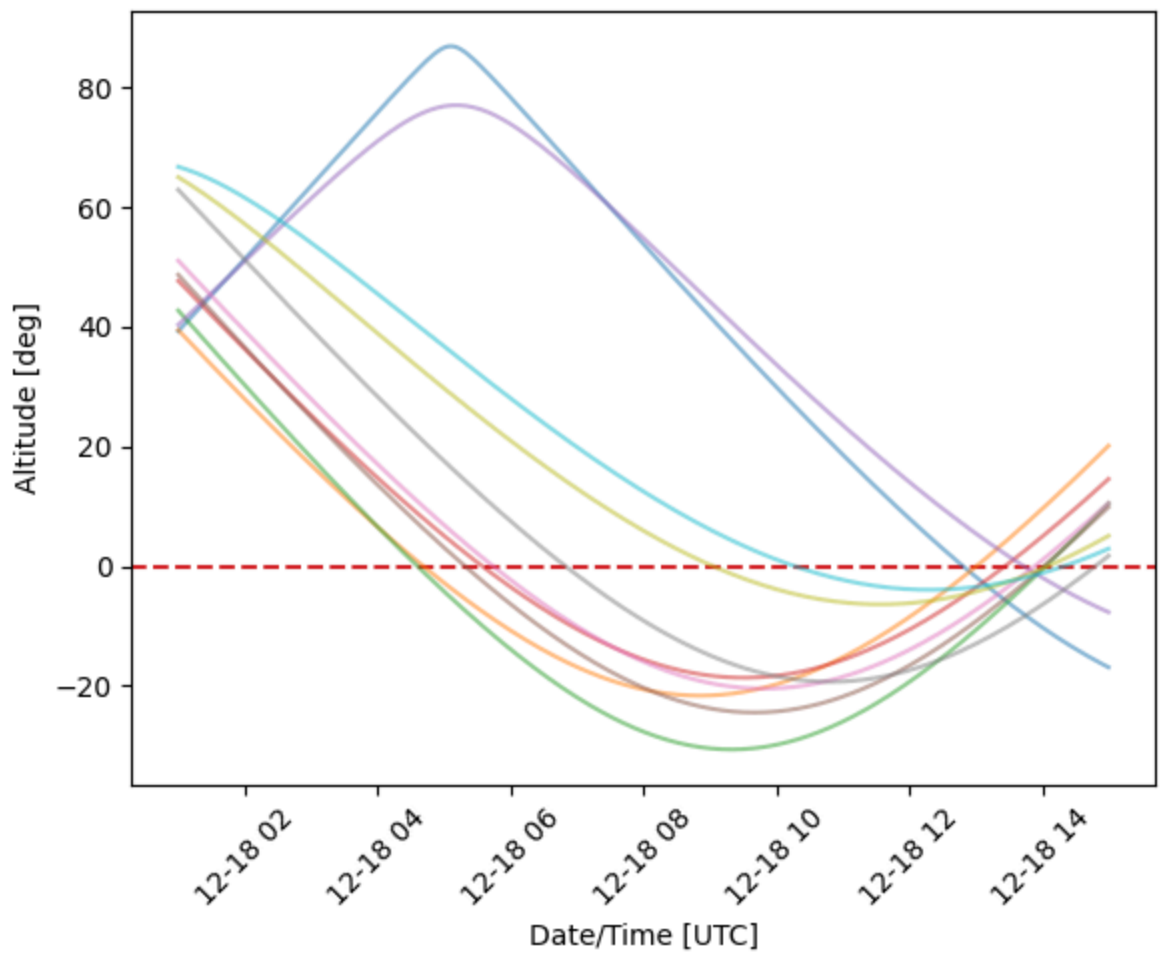
```
(474, 256)
```

We therefore want to produce a two-dimensional coordinate object that is indexed along one axis by the open cluster index, and along other axis by the time index. The `astropy.coordinates` transformation machinery supports array-like broadcasting, so we can do this by creating new, unmatched, length-1 axes on both the open clusters `SkyCoord` object and the `AltAz` frame using `numpy.newaxis` ([doc]()):

```
open_cluster_altaz = open_cluster_c[:, np.newaxis].transform_to(altaz[np.newaxis])
```

Let's now over-plot the trajectories for the first 10 open clusters in the catalog:

```
plt.figure(figsize=(6, 5))
plt.plot(time_grid.datetime, open_cluster_altaz[:10].alt.degree.T, marker="", alpha=
plt.axhline(0, zorder=-10, linestyle="--", color="tab:red")
plt.xlabel("Date/Time [UTC]")
plt.ylabel("Altitude [deg]")
plt.setp(plt.gca().xaxis.get_majorticklabels(), rotation=45)
plt.tight_layout()
```



From this, we can see that only two of the clusters in this batch seem to be easily observable.

Potential Caveats

Transformations between some reference frames require knowing more information about a source. For example, the transformation from ICRS to Galactic coordinates (as demonstrated above) amounts to a 3D rotation, but no change of origin. This transformation is therefore supported for any spherical position (with or without distance information). However, some transformations include a change of origin, and therefore require that the source data (i.e., the `SkyCoord` object) has defined distance information. For example, for a `SkyCoord` with only sky position, we can transform from the ICRS to the FK5 coordinate system:

```
icrs_c = coord.SkyCoord(150.4 * u.deg, -11 * u.deg)
icrs_c.transform_to(coord.FK5())
```

```
<SkyCoord (FK5: equinox=J2000.000): (ra, dec) in deg
    (150.40000705, -11.00000493)>
```

However, we would **NOT** be able to transform this position to the `Galactocentric` frame (docs), because this transformation involves a shift of origin from the solar system barycenter to the Galactic center:

```
# This cell will raise an exception and will not work (you will see a `ConvertError

icrs_c.transform_to(coord.Galactocentric())
```

```
---------------------------------------------------------------------------
ConvertError                              Traceback (most recent call last)
Cell In[35], line 3
      1 # This cell will raise an exception and will not work (you will see a `Conve
----> 3 icrs_c.transform_to(coord.Galactocentric())

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    551 generic_frame = GenericFrame(frame_kwargs)
    553 # Do the transformation, returning a coordinate frame of the desired
    554 # final type (not generic).
--> 555 new_coord = trans(self.frame, generic_frame)
    557 # Finally make the new SkyCoord object from the `new_coord` and
    558 # remaining frame_kwargs that are not frame_attributes in `new_coord`.
    559 for attr in set(new_coord.frame_attributes) & set(frame_kwargs.keys()):

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    110             frattrs[inter_frame_attr_nm] = attr
    112     curr_toframe = t.tosys(**frattrs)
--> 113     curr_coord = t(curr_coord, curr_toframe)
    115 # this is safe even in the case where self.transforms is empty, because
    116 # coordinate objects are immutable, so copying is not needed
    117 return curr_coord

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    204 def __call__(self, fromcoord, toframe):
--> 205     params = self._affine_params(fromcoord, toframe)
    206     newrep = self._apply_transform(fromcoord, *params)
    207     return toframe.realize_frame(newrep)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    258 def _affine_params(self, fromcoord, toframe):
--> 259     return self.transform_func(fromcoord, toframe)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    590 @frame_transform_graph.transform(AffineTransform, ICRS, Galactocentric)
    591 def icrs_to_galactocentric(icrs_coord, galactocentric_frame):
--> 592     _check_coord_repr_diff_types(icrs_coord)
    593     return get_matrix_vectors(galactocentric_frame)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    569 def _check_coord_repr_diff_types(c):
    570     if isinstance(c.data, r.UnitSphericalRepresentation):
--> 571         raise ConvertError(
    572             "Transforming to/from a Galactocentric frame requires a 3D coord
    573             " (angle, angle, distance) or (x, y, z)."
    574         )
    576     if "s" in c.data.differentials and isinstance(
    577         c.data.differentials["s"],
    578         (
   (...)    582         ),
    583     ):
    584         raise ConvertError(
    585             "Transforming to/from a Galactocentric frame requires a 3D veloc
```

```
            " proper motion components and radial velocity."
        )

ConvertError: Transforming to/from a Galactocentric frame requires a 3D coordinate,
```

In this tutorial, we have introduced the key concepts that underly the `astropy.coordinates` framework: coordinate component formats, representations, and frames. We demonstrated how to change the representation of a `SkyCoord` object (e.g., Cartesian to Cylindrical). We then introduce the concept of coordinate frames and frame transformations.

Exercises

Przybylski's star or HD101065 is in the southern constellation of Centaurus with a right ascension of 174.4040348 degrees and a declination of -46.70953633 degrees. Create a `SkyCoord` object of its sky position.

If the distance to Przybylski's star is 108.4 pc, retrieve a 3D Cartesian representation. (Hint: we did this earlier in the tutorial and it may help to create a new 3D `SkyCoord` object.)

Imagine it is May 2018, and you would like to take an observation of HD 101065 from Greenwich Royal Observatory. Use astropy.coordinates to figure out if you can observe the star that month (hint: use the `EarthLocation` class). You can use any time and date of that month for your timeframe.

Authors

Adrian Price-Whelan, Saima Siddiqui, Luthien Liu, Zihao Chen

Learning Goals

- Introduce how to represent and transform velocity data in `SkyCoord` objects
- Demonstrate how to predict the position of a star at a time using its proper motion

Keywords

coordinates, OOP, astroquery, gaia

Summary

In the previous tutorial in this series, we showed how astronomical *positional* coordinates can be represented and transformed in Python using the `SkyCoord` object (docs). Many sources, especially stars (thanks to the Gaia mission), have measured velocities or measured components of their velocity (e.g., just proper motion, or just radial velocity).

In this tutorial, we will explore how the `astropy.coordinates` package can be used to represent and transform astronomical coordinates that have associated velocity data. You may find it helpful to keep the Astropy documentation for the coordinates package open alongside this tutorial for reference or additional reading. In the text below, you may also see some links that look like (docs). These links will take you to parts of the documentation that are directly relevant to the cells from which they link.

Note: This is the 3rd tutorial in a series of tutorials about astropy.coordinates. If you are new to astropy.coordinates, you may want to start from the beginning or an earlier tutorial.

Imports

We start by importing some general packages we will need below:

```
import warnings

import matplotlib.pyplot as plt

%matplotlib inline
import numpy as np

from astropy import units as u
from astropy.coordinates import SkyCoord, Distance, Galactic
import astropy.coordinates as coord
from astropy.io import fits
from astropy.table import QTable
from astropy.time import Time
from astropy.utils.data import download_file
from astropy.wcs import WCS

from astroquery.gaia import Gaia
```

More Than Sky Positions: Including Velocity Data in `SkyCoord`

As we have seen in the previous tutorials, the `SkyCoord` object can be used to store scalars or arrays of positional coordinate information and supports transforming between different coordinate frames and representations. But `astropy.coordinates` also supports representing and transforming *velocity* information along with positions (docs).

Passing Velocity Data into `SkyCoord`

Velocity components are passed in to `SkyCoord` in the same way that positional components are specified: As arguments to the `SkyCoord` class. For example, to create a `SkyCoord` to represent a sky position and a proper motion in the (default) ICRS coordinate frame, in addition to the position components `ra`, `dec`, we can pass in values for the proper motion components `pm_ra_cosdec`, `pm_dec` ("pm" for "proper motion"):

```
SkyCoord(
    ra=10 * u.deg,
    dec=20 * u.deg,
    pm_ra_cosdec=1 * u.mas / u.yr,
    pm_dec=2 * u.mas / u.yr,
)
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (10., 20.)
 (pm_ra_cosdec, pm_dec) in mas / yr
    (1., 2.)>
```

Here, you may notice that the proper motion in right ascension has "cosdec" in the name: This is to explicitly note that the input here is expected to be the proper motion scaled by the cosine of the declination, which accounts for the fact that a change in longitude (right ascension) has different physical length at different latitudes (declinations).

Like the examples in previous tutorials demonstrated for positional coordinates, we can also create an array-valued `SkyCoord` object by passing in arrays of data for all of the components. In this case, each value in the inputed array represents a quantity of an object among large data set. This method would be beneficial when dealing with large number of star collections like a star cluster:

```
SkyCoord(
    ra=np.linspace(0, 10, 5) * u.deg,
    dec=np.linspace(5, 20, 5) * u.deg,
    pm_ra_cosdec=np.linspace(-5, 5, 5) * u.mas / u.yr,
    pm_dec=np.linspace(-5, 5, 5) * u.mas / u.yr,
)
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    [( 0. ,  5.  ), ( 2.5,  8.75), ( 5. , 12.5 ), ( 7.5, 16.25),
     (10. , 20.  )]
 (pm_ra_cosdec, pm_dec) in mas / yr
    [(-5. , -5. ), (-2.5, -2.5), ( 0. ,  0. ), ( 2.5,  2.5), ( 5. ,  5. )]>
```

However, for some of the examples below we will continue to use scalar values for brevity.

You can also specify radial velocity data with the `radial_velocity` argument:

```
velocity_coord = SkyCoord(
    ra=10 * u.deg,
    dec=20 * u.deg,
    pm_ra_cosdec=1 * u.mas / u.yr,
    pm_dec=2 * u.mas / u.yr,
    radial_velocity=100 * u.km / u.s,
)
velocity_coord
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (10., 20.)
 (pm_ra_cosdec, pm_dec, radial_velocity) in (mas / yr, mas / yr, km / s)
    (1., 2., 100.)>
```

The component data can then be accessed using the same names used to pass in the velocity components:

```
velocity_coord.pm_ra_cosdec
```

$$1\ \frac{\mathrm{mas}}{\mathrm{yr}}$$

```
velocity_coord.radial_velocity
```

$$100\ \frac{\mathrm{km}}{\mathrm{s}}$$

A `SkyCoord` object with velocity data can be transformed to other frames just like the position-only coordinate objects we used in the previous tutorials:

```
velocity_coord_gal = velocity_coord.transform_to(Galactic())
velocity_coord_gal
```

```
<SkyCoord (Galactic): (l, b) in deg
    (119.26936774, -42.79039286)
 (pm_l_cosb, pm_b, radial_velocity) in (mas / yr, mas / yr, km / s)
    (1.11917063, 1.93583499, 100.)>
```

Note that, like the position components, which change from `ra`, `dec` to `l`, `b`, the proper motion component names have changed to reflect naming conventions for the component names in a given frame: `pm_ra_cosdec` and `pm_dec` have become `pm_l_cosb` and `pm_b`:

```
velocity_coord_gal.pm_l_cosb
```

$$1.1191706\ \frac{\mathrm{mas}}{\mathrm{yr}}$$

```
velocity_coord_gal.pm_b
```

$$1.935835\ \frac{\mathrm{mas}}{\mathrm{yr}}$$

An important caveat to note when transforming a `SkyCoord` object with velocity data is that some reference frames require knowing the distances, or the full velocity vectors (i.e. proper motion components and radial velocity) in order to transform the velocities correctly. For example, a

`SkyCoord` with only sky position and proper motion data **CANNOT** be transformed to a frame with a positional or velocity offset, such as the `Galactocentric` frame (docs)

```
# This cell will NOT work (you will receive an ConvertError warning) - this is expec

test_coord_1 = SkyCoord(
    ra=10 * u.deg,
    dec=20 * u.deg,
    pm_ra_cosdec=1 * u.mas / u.yr,
    pm_dec=2 * u.mas / u.yr,
)

test_coord_1.transform_to(coord.Galactocentric())
```

```
---------------------------------------------------------------------------
ConvertError                              Traceback (most recent call last)
Cell In[10], line 10
      1 # This cell will NOT work (you will receive an ConvertError warning) - this
      3 test_coord_1 = SkyCoord(
      4     ra=10 * u.deg,
      5     dec=20 * u.deg,
      6     pm_ra_cosdec=1 * u.mas / u.yr,
      7     pm_dec=2 * u.mas / u.yr,
      8 )
---> 10 test_coord_1.transform_to(coord.Galactocentric())

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    551 generic_frame = GenericFrame(frame_kwargs)
    553 # Do the transformation, returning a coordinate frame of the desired
    554 # final type (not generic).
--> 555 new_coord = trans(self.frame, generic_frame)
    557 # Finally make the new SkyCoord object from the `new_coord` and
    558 # remaining frame_kwargs that are not frame_attributes in `new_coord`.
    559 for attr in set(new_coord.frame_attributes) & set(frame_kwargs.keys()):

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    110             frattrs[inter_frame_attr_nm] = attr
    112     curr_toframe = t.tosys(**frattrs)
--> 113     curr_coord = t(curr_coord, curr_toframe)
    115 # this is safe even in the case where self.transforms is empty, because
    116 # coordinate objects are immutable, so copying is not needed
    117 return curr_coord

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    204 def __call__(self, fromcoord, toframe):
--> 205     params = self._affine_params(fromcoord, toframe)
    206     newrep = self._apply_transform(fromcoord, *params)
    207     return toframe.realize_frame(newrep)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    258 def _affine_params(self, fromcoord, toframe):
--> 259     return self.transform_func(fromcoord, toframe)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    590 @frame_transform_graph.transform(AffineTransform, ICRS, Galactocentric)
    591 def icrs_to_galactocentric(icrs_coord, galactocentric_frame):
--> 592     _check_coord_repr_diff_types(icrs_coord)
    593     return get_matrix_vectors(galactocentric_frame)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    569 def _check_coord_repr_diff_types(c):
    570     if isinstance(c.data, r.UnitSphericalRepresentation):
--> 571         raise ConvertError(
    572             "Transforming to/from a Galactocentric frame requires a 3D coord
    573             " (angle, angle, distance) or (x, y, z)."
    574         )
    576     if "s" in c.data.differentials and isinstance(
```

```
    577         c.data.differentials["s"],
    578         (
   (...)    582         ),
    583     ):
    584         raise ConvertError(
    585             "Transforming to/from a Galactocentric frame requires a 3D veloc
    586             " proper motion components and radial velocity."
    587         )

ConvertError: Transforming to/from a Galactocentric frame requires a 3D coordinate,
```

In order to transform to the `Galactocentric` frame, both distance and radial velocity of the object are required.

```
test_coord_2 = SkyCoord(
    ra=10 * u.deg,
    dec=20 * u.deg,
    distance=10 * u.pc,
    pm_ra_cosdec=1 * u.mas / u.yr,
    pm_dec=2 * u.mas / u.yr,
    radial_velocity=100 * u.km / u.s,
)

test_coord_2.transform_to(coord.Galactocentric())
```

```
<SkyCoord (Galactocentric: galcen_coord=<ICRS Coordinate: (ra, dec) in deg
    (266.4051, -28.936175)>, galcen_distance=8.122 kpc, galcen_v_sun=(12.9, 245.6, 7
    (-8125.57861635, 6.40155855, 14.01603221)
 (v_x, v_y, v_z) in km / s
    (-23.22908903, 309.64402716, -59.99213843)>
```

Evolving Coordinate Positions Between Epochs

For nearby or fast-moving stars, a star's position could change appreciably between two well-spaced observations of the source. For such cases, it might be necessary to compute the position of the star at a given time using the proper motion or velocity of the star. Let's demonstrate this idea by comparing the sky position of a source as measured by *Gaia* Data Release 2 (given at the epoch J2015.5) to an image near this source from the Digitized Sky Survey (DSS; digital scans of photographic plates observed in the 1950s).

From previous astrometric measurements, we know that the star HD 219829 has very large proper motion: Close to 0.5 arcsec/year! Between the DSS and *Gaia*, we therefore expect that the position of

the star has changed by about 0.5 arcmin. Let's see if this is the case!

To start, we will query the *Gaia* catalog to retrieve data for this star (skip the cell below if you do not have an internet connection - we have provided the table locally as well). We use a large search radius so many sources will be returned

```
# Skip this cell if you are not connected to the internet
gaia_tbl = Gaia.query_object(SkyCoord.from_name("HD 219829"), radius=1 * u.arcmin)
```

```
# the .read() below produces some warnings that we can safely ignore
with warnings.catch_warnings():
    warnings.simplefilter("ignore", UserWarning)

    gaia_tbl = QTable.read("HD_219829_query_results.ecsv")
```

We know that HD 219829 will be the brightest source in this small region, so we can extract the row with the smallest G-band magnitude. Let's check the proper motion values for this source to make sure that they are large:

```
hd219829_row = gaia_tbl[gaia_tbl["phot_g_mean_mag"].argmin()]
hd219829_row["source_id", "pmra", "pmdec"]
```

Row index=2

source_id	pmra	pmdec
	mas / yr	mas / yr
int64	float64	float64
2661015540210781568	483.4165901889734	-114.8633971841528

Indeed, it looks like this is our source! Let's construct a `SkyCoord` object for this source using the data from the *Gaia* archive:

*Note about the Gaia catalog proper motion column names: The names in the Gaia archive and other repositories containing Gaia data give Right Ascension proper motion values simply as "pmra". These components implicitly contain the `cos(dec)` term, so we do **not** have to modify these values in order to pass them in to `SkyCoord` as `pm_ra_cosdec`*

```
hd219829_coord = SkyCoord(
    ra=hd219829_row["ra"],
    dec=hd219829_row["dec"],
    distance=Distance(parallax=hd219829_row["parallax"]),
    pm_ra_cosdec=hd219829_row["pmra"],
    pm_dec=hd219829_row["pmdec"],
    obstime=Time(hd219829_row["ref_epoch"], format="jyear"),
)

hd219829_coord
```

```
<SkyCoord (ICRS): (ra, dec, distance) in (deg, deg, pc)
    (349.72896716, 5.40511585, 34.47896069)
 (pm_ra_cosdec, pm_dec) in mas / yr
    (483.41659019, -114.86339718)>
```

We now have a `SkyCoord` representation of the position and proper motion of the star HD 219829 as measured by *Gaia* and reported at the epoch J2015.5. What does this mean exactly? *Gaia* actually measures the (time-dependent) position of a star every time it scans the part of the sky that contains the source, and this is how *Gaia* is able to measure proper motions of stars. However, if every star is moving and changing its sky positions, how do we ever talk about “the sky position” of a star as opposed to “the sky trajectory of a star”?! The key is that catalogs often only report the position of a source at some reference epoch. For a survey that only observes the sky once or a few times (e.g., SDSS or 2MASS), this reference epoch might be “the time that the star was observed.” But for a mission like *Gaia*, which scans the sky many times, they perform astrometric fits to the individual position measurements, which allow them to measure the parallax, proper motion, and the reference position at a reference time for each source. For *Gaia* data release 2, the reference time is J2015.5, and the sky positions (and other quantities) reported in the catalog for each source are at this epoch.

In `SkyCoord`, we specify the “epoch” of an observation using the `obstime` argument, as we did above. Now that we have a coordinate object for HD 219829, let’s now compare the position of the star as measured by *Gaia* to its apparent position in an image from the DSS. Let’s now query the DSS to retrieve a FITS image of the field around this star, using the STSCI DSS image cutout service. Skip the cell below if you do not have an internet connection (we have provided the image locally as well):

```
# Skip this cell if you are not connected to the internet
dss_cutout_filename = download_file(
    f"http://archive.stsci.edu/cgi-bin/dss_search?"
    f"f=FITS&ra={hd219829_coord.ra.degree}&dec={hd219829_coord.dec.degree}"
    f"&width=4&height=4"
)  # width/height in arcmin
```

```
dss_cutout_filename = "dss_hd219829.fits"
```

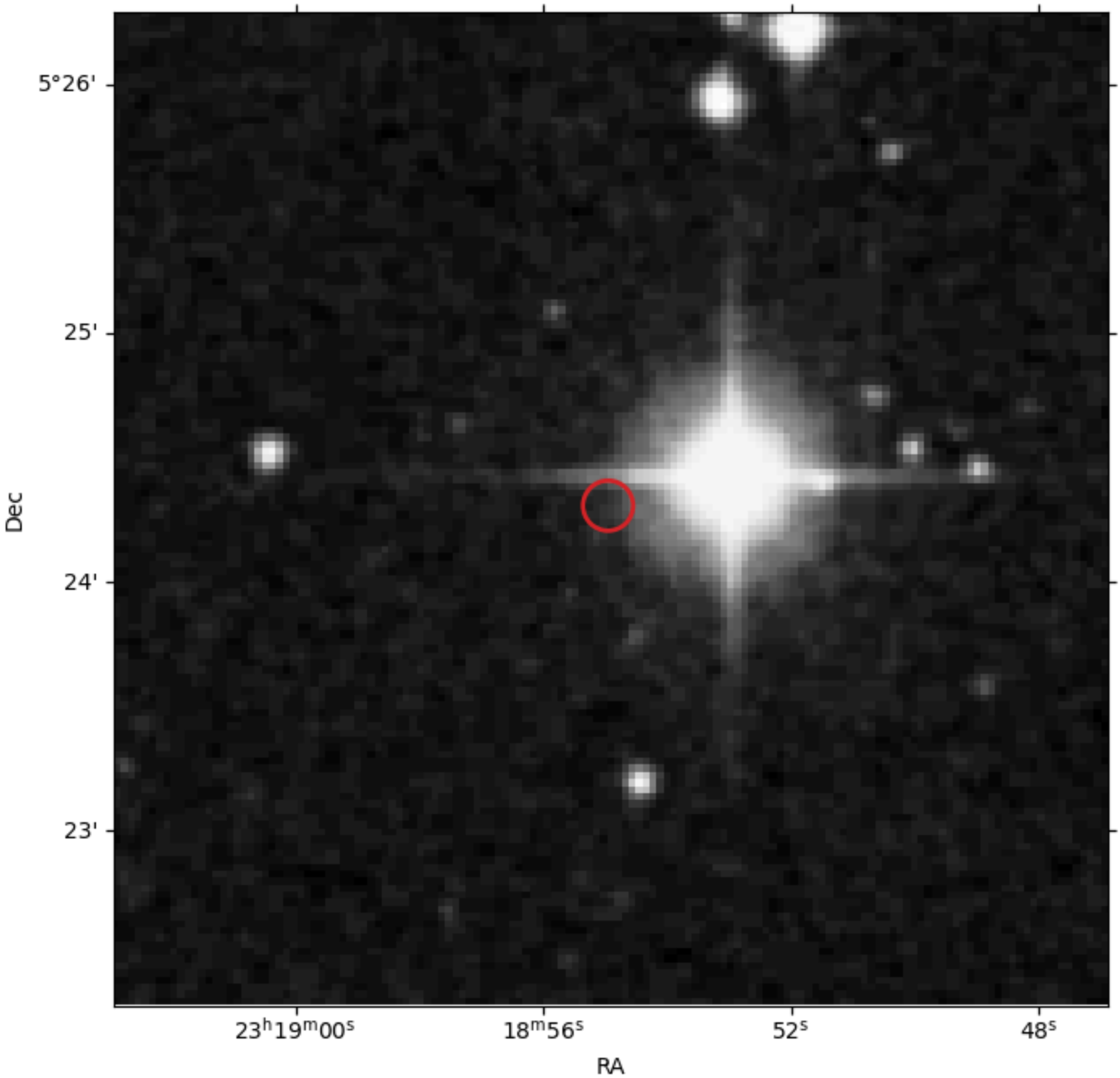
We can now load the FITS image of the cutout and use `astropy.visualization` to display the image using its World Coordinate System (WCS) info (docs). By passing in the WCS information (included in the FITS cutout header), we can over-plot a marker for the *Gaia*-measured sky position of HD 219829:

(If you are unfamiliar with the usage of FITS header and FITS image, check out these 2 tutorials having detailed explanation on these 2 topics: FITS-Header and FITS-Image.)

```
hdu = fits.open(dss_cutout_filename)[0]
wcs = WCS(hdu.header)

fig, ax = plt.subplots(1, 1, figsize=(8, 8), subplot_kw=dict(projection=wcs))
ax.imshow(hdu.data, origin="lower", cmap="Greys_r")
ax.set_xlabel("RA")
ax.set_ylabel("Dec")
ax.set_autoscale_on(False)

ax.scatter(
    hd219829_coord.ra.degree,
    hd219829_coord.dec.degree,
    s=500,
    transform=ax.get_transform("world"),
    facecolor="none",
    linewidth=2,
    color="tab:red",
)
```

```
WARNING: FITSFixedWarning: 'datfix' made the change 'Set MJD-OBS to 33867.416667 fro
```

```
<matplotlib.collections.PathCollection at 0x7ff725c361b0>
```



The brighest star (as observed by DSS) in this image is our target, and the red circle is where *Gaia* observed this star. As we excpected, it has moved quite a bit since the 1950's! We can account for this motion and predict the position of the star at around the time the DSS plate was observed. Let's assume that this plate was observed in 1950 exactly (this is not strictly correct, but should get us close enough).

To account for the proper motion of the source and evolve the position to a new time, we can use the `SkyCoord.apply_space_motion()` method (docs). Because we defined the `obstime` when we defined the coordinate object for HD 219829, for example:

```
hd219829_coord.obstime
```

```
<Time object: scale='tt' format='jyear' value=2015.5>
```

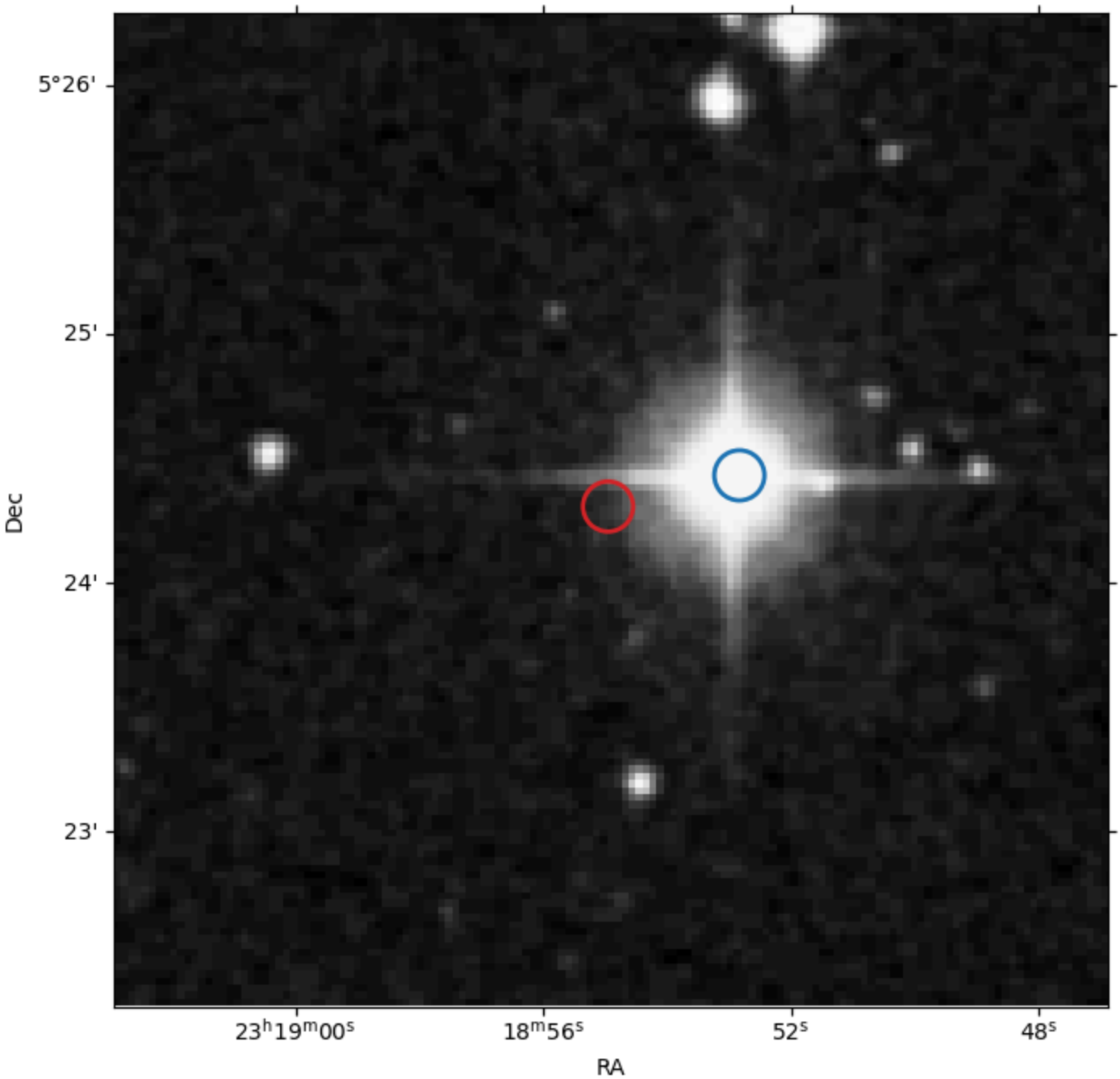
We can now use `apply_space_motion()` by passing in a new time, `new_obstime`, to compute the coordinates at:

```
# this produces some warnings that we can safely ignore
with warnings.catch_warnings():
    warnings.simplefilter("ignore", UserWarning)

    hd219829_coord_1950 = hd219829_coord.apply_space_motion(new_obstime=Time("J1950"
```

Let's now plot our predicted position for this source as it would appear in 1950 based on the *Gaia* position and proper motion:

```python
fig, ax = plt.subplots(1, 1, figsize=(8, 8), subplot_kw=dict(projection=wcs))
ax.imshow(hdu.data, origin="lower", cmap="Greys_r")
ax.set_xlabel("RA")
ax.set_ylabel("Dec")
ax.set_autoscale_on(False)

ax.scatter(
    hd219829_coord.ra.degree,
    hd219829_coord.dec.degree,
    s=500,
    transform=ax.get_transform("world"),
    facecolor="none",
    linewidth=2,
    color="tab:red",
)

# Plot the predicted (past) position:
ax.scatter(
    hd219829_coord_1950.ra.degree,
    hd219829_coord_1950.dec.degree,
    s=500,
    transform=ax.get_transform("world"),
    facecolor="none",
    linewidth=2,
    color="tab:blue",
)
```

```
<matplotlib.collections.PathCollection at 0x7ff724e90620>
```



The red circle is the same as in the previous image and shows the position of the source in the *Gaia* catalog (in 2015.5). The blue circle shows our prediction for the position of the source in 1950 - this looks much closer to where the star is in the DSS image!

In this tutorial, we have introduced how to store and transform velocity data along with positional data in `astropy.coordinates`. We also demonstrated how to use the velocity of a source to predict its position at an earlier or later time.

Exercises

Lalande 21185 is the brightest red dwarf star in the northern hemisphere and has a pretty high proper motion. Use the Gaia archive (https://gea.esac.esa.int/archive/) to find values and create a `SkyCoord` object for Lalande 21185. (Hint: earlier in the tutorial, we extracted information from a Gaia table and mentioned which Gaia column names match with our postition components).

Authors

Adrian Price-Whelan

Learning Goals

- Demonstrate how to retrieve a catalog from Vizier using astroquery
- Show how to perform positional cross-matches between catalogs of sky coordinates

Keywords

coordinates, OOP, astroquery, gaia

Summary

In the previous tutorials in this series, we introduced many of the key concepts underlying how to represent and transform astronomical coordinates using `astropy.coordinates`, including how to work with both position and velocity data within the coordinate objects.

In this tutorial, we will explore how the `astropy.coordinates` package can be used to cross-match two catalogs that contain overlapping sources that may have been observed at different times. You may find it helpful to keep the Astropy documentation for the coordinates package open alongside this tutorial for reference or additional reading. In the text below, you may also see some links that look like (docs). These links will take you to parts of the documentation that are directly relevant to the cells from which they link.

Note: This is the 4th tutorial in a series of tutorials about astropy.coordinates. If you are new to astropy.coordinates, you may want to start from the beginning or an earlier tutorial.

Imports

We start by importing some general packages that we will need below:

```
import warnings

import matplotlib.pyplot as plt

%matplotlib inline
import numpy as np

from astropy import units as u
from astropy.coordinates import SkyCoord, Distance
from astropy.table import QTable
from astropy.time import Time

from astroquery.vizier import Vizier
```

Cross-matching and comparing catalogs

In this tutorial, we are going to return to a set of data that we downloaded from the *Gaia* archive back in Tutorial 1 of this series.

Let's recap what we did in that tutorial: We defined a `SkyCoord` object to represent the center of an open cluster (NGC 188), we queried the *Gaia* DR2 catalog to select stars that are close (on the sky) to the center of the cluster, and we used the parallax values from *Gaia* to select stars that are near NGC 188 in 3D position. Here, we will briefly reproduce those selections so that we can start here with a catalog of sources that are likely members of NGC 188 (see Tutorial 1 for more information):

```python
ngc188_table = QTable.read("gaia_results.fits")
ngc188_table = ngc188_table[ngc188_table["parallax"] > 0.25 * u.mas]

ngc188_center_3d = SkyCoord(
    12.11 * u.deg,
    85.26 * u.deg,
    distance=1.96 * u.kpc,
    pm_ra_cosdec=-2.3087 * u.mas / u.yr,
    pm_dec=-0.9565 * u.mas / u.yr,
)

# Deal with masked quantity data in a backwards-compatible way:
parallax = ngc188_table["parallax"]
if hasattr(parallax, "mask"):
    parallax = parallax.filled(np.nan)

velocity_data = {
    "pm_ra_cosdec": ngc188_table["pmra"],
    "pm_dec": ngc188_table["pmdec"],
    "radial_velocity": ngc188_table["radial_velocity"],
}
for k, v in velocity_data.items():
    if hasattr(v, "mask"):
        velocity_data[k] = v.filled(0.0)
    velocity_data[k][np.isnan(velocity_data[k])] = 0.0

ngc188_coords_3d = SkyCoord(
    ra=ngc188_table["ra"],
    dec=ngc188_table["dec"],
    distance=Distance(parallax=parallax),
    obstime=Time("J2015.5"),
    **velocity_data,
)

sep3d = ngc188_coords_3d.separation_3d(ngc188_center_3d)
pm_diff = np.sqrt(
    (ngc188_table["pmra"] - ngc188_center_3d.pm_ra_cosdec) ** 2
    + (ngc188_table["pmdec"] - ngc188_center_3d.pm_dec) ** 2
)

ngc188_members_mask = (sep3d < 50 * u.pc) & (pm_diff < 1.5 * u.mas / u.yr)
ngc188_members = ngc188_table[ngc188_members_mask]
ngc188_members_coords = ngc188_coords_3d[ngc188_members_mask]
len(ngc188_members)
```

```
265
```

From the selections above, the table `ngc188_members` and the `SkyCoord` instance `ngc188_members_coords` contain 216 sources that, based on their 3D positions and proper motions,

are consistent with being members of the open cluster NGC 188.

Let's assume that we now want to cross-match our catalog of candidate members of NGC 188 — here, based on *Gaia* data — to some other catalog. In this tutorial, we will demonstrate how to manually cross-match these *Gaia* sources with the 2MASS photometric catalog to retrieve infrared magnitudes for these stars, and then we will plot a color–magnitude diagram. To do this, we first need to query the 2MASS catalog to retrieve all sources in a region around the center of NGC 188, as we did for *Gaia*. Here, we will also take into account the fact that the *Gaia* data release 2 reference epoch is J2015.5, whereas the 2MASS coordinates are likely reported at their time of observation (in the late 1990's).

Note that some data archives, like the Gaia science archive, support running cross-matches at the database level and even support epoch propagation. If you need to perform a large cross-match, it will be much more efficient to use these services!

We will again use `astroquery` to execute this query. This will again require an internet connection, but we have included the results of this query in a file along with this notebook in case you are not connected to the internet. To query 2MASS, we will use the `astroquery.vizier` module (docs) to run a cone search centered on the sky position of NGC 188 with a search radius of 0.5º:

```
# NOTE: skip this cell if you do not have an internet connection

# II/246 is the catalog name for the main 2MASS photometric catalog
v = Vizier(catalog="II/246", columns=["*", "Date"])
v.ROW_LIMIT = -1

result = v.query_region(ngc188_center_3d, radius=0.5 * u.deg)
tmass_table = result[0]
```

Alternatively, we can read the 2MASS table provided along with this tutorial:

```
# the .read() below produces some warnings that we can safely ignore
with warnings.catch_warnings():
    warnings.simplefilter("ignore", UserWarning)

    tmass_table = QTable.read("2MASS_results.ecsv")
```

As with the *Gaia* results table, we can now create a single `SkyCoord` object to represent all of the sources returned from our query to the 2MASS catalog. Let's look at the column names in this table by displaying the first few rows:

```
tmass_table[:3]
```

QTable length=3

Date	RAJ2000	DEJ2000	_2MASS	Jmag	e_Jmag	Hmag	e_Hmag	Km
	deg	deg		mag	mag	mag	mag	m
str10	float64	float64	str16	float32	float32	float32	float32	float
1999-10-19	9.633532	84.808350	00383204+8448300	13.079	0.030	12.742	0.033	12.7
1999-10-19	8.562472	84.873627	00341499+8452250	14.459	0.029	14.103	0.041	14.1
1999-10-19	8.645318	84.868584	00343487+8452069	10.356	0.026	10.057	0.032	10.0

From looking at the column names, the two relevant sky coordinate columns are `RAJ2000` for `ra` and `DEJ2000` for `dec`:

```
tmass_coords = SkyCoord(tmass_table["RAJ2000"], tmass_table["DEJ2000"])
len(tmass_coords)
```

```
5014
```

Note also that the table contains a “Date” column that specifies the epoch of the coordinates. Are all of these epochs the same?

```
np.unique(tmass_table["Date"])
```

<Column name='Date' dtype='str10' description='(date) Observation date' length=1>

1999-10-19

It looks like all of the sources in our 2MASS table have the same epoch, so let’s create an `astropy.time.Time` object to represent this date:

```
tmass_epoch = Time(np.unique(tmass_table["Date"]))
```

We now want to cross-match our *Gaia*-selected candidate members of NGC 188, `ngc_members_coords`, with this table of photometry from 2MASS. However, as noted previously, the *Gaia* coordinates are given at a different epoch J2015.5, which is nearly ~16 years after the 2MASS epoch of the data we downloaded (1999-10-19 or roughly J1999.88). We will therefore first use the `SkyCoord.apply_space_motion()` method (docs) to transform the *Gaia* positions back to the 2MASS epoch before we do the cross-match:

```
# you can ignore the warning raised here
ngc188_members_coords_1999 = ngc188_members_coords.apply_space_motion(tmass_epoch)
```

The object `ngc188_members_coords_1999` now contains the coordinate information for our *Gaia*-selected members of NGC 188, as we think they would appear if observed on 1999-10-19.

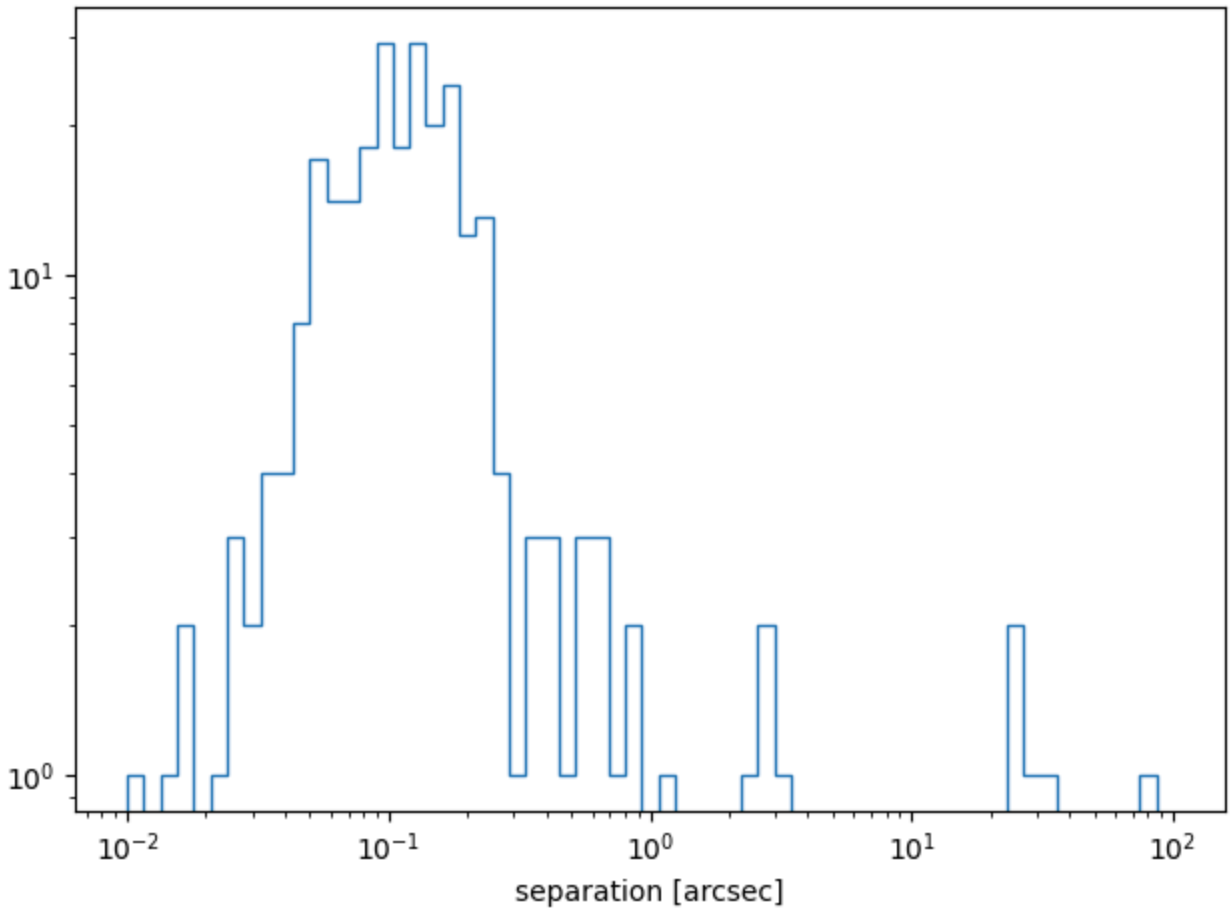
We can now use the `SkyCoord.match_to_catalog_sky` method to match these two catalogs (docs), using the `ngc188_members_coords_1999` as our NGC 188 members coordinates.

Note that order matters with this method: Here we will match *Gaia* to 2MASS. `SkyCoord.match_to_catalog_sky` returns three objects: (1) the indices into `tmass_coords` that get the closest matches in `ngc188_members_coords_1999`, (2) the angular separation between each `ngc188_members_coords_1999` coordinate and the closest source in `tmass_coords`, and (3) the 3D distance between each `ngc188_members_coords_1999` coordinate and the closest source in `tmass_coords`. Here, the 3D distances will not be useful because the 2MASS coordinates do not have associated distance information, so we will ignore these quantities:

```
idx_gaia, sep2d_gaia, _ = ngc188_members_coords_1999.match_to_catalog_sky(tmass_coo
```

Let's now look at the distribution of separations (in arcseconds) for all of the cross-matched sources:

```
plt.hist(sep2d_gaia.arcsec, histtype="step", bins=np.logspace(-2, 2.0, 64))
plt.xlabel("separation [arcsec]")
plt.xscale("log")
plt.yscale("log")
plt.tight_layout()
```



From this, it looks like all of sources in our *Gaia* NGC 188 member list cross-match to another sources within a few arcseconds, so these all seem like they are correctly matches to a 2MASS source!

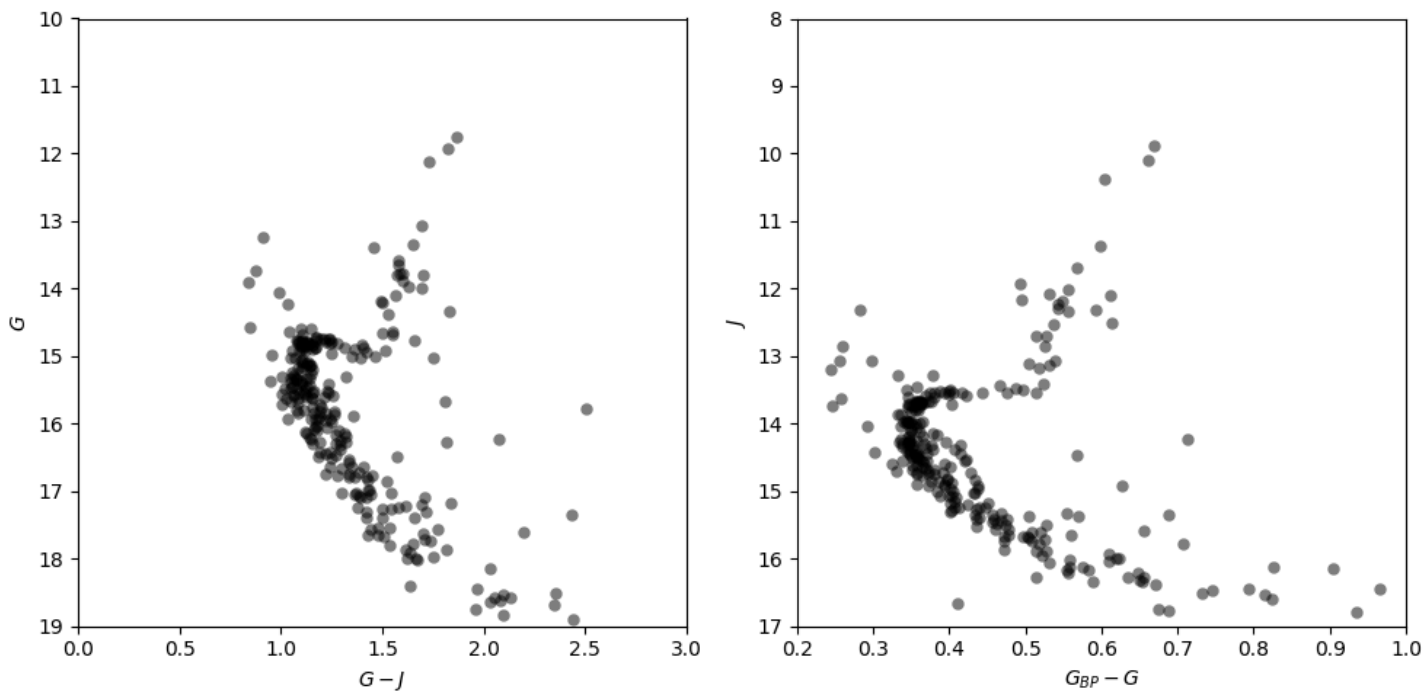
```
(sep2d_gaia < 2 * u.arcsec).sum(), len(ngc188_members)
```

```
(np.int64(256), 265)
```

With our cross-match done, we can now make `Gaia`+2MASS color–magnitude diagrams of our candidate NGC 188 members using the information returned by the cross-match:

```
Jmag = tmass_table["Jmag"][idx_gaia]  # note that we use the index array returned al
Gmag = ngc188_members["phot_g_mean_mag"]
Bmag = ngc188_members["phot_bp_mean_mag"]
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

ax = axes[0]
ax.scatter(Gmag - Jmag, Gmag, marker="o", color="k", linewidth=0, alpha=0.5)
ax.set_xlabel("$G - J$")
ax.set_ylabel("$G$")
ax.set_xlim(0, 3)
ax.set_ylim(19, 10)  # backwards because magnitudes!

ax = axes[1]
ax.scatter(Bmag - Gmag, Jmag, marker="o", color="k", linewidth=0, alpha=0.5)
ax.set_xlabel("$G_{BP} - G$")
ax.set_ylabel("$J$")
ax.set_xlim(0.2, 1)
ax.set_ylim(17, 8)  # backwards because magnitudes!

fig.tight_layout()
```



Those both look like color-magnitude diagrams of a main sequence + red giant branch of an intermediate-age stellar cluster, so it looks like our selection and cross-matching has worked!

For more on what matching options are available, check out the [separation and matching section of the Astropy documentation](). Or for more on what you can do with `SkyCoord`, see [its API documentation]().